

Yazılım Hata Ayıklama Teknolojilerinin Temelleri ve Evrimi: Geleneksel Yöntemlerden ve Hata Konumlandırılmadan Büyük Dil Modelleri ve Etmen Tabanlı Hata Ayıklamaya

Konu Başlığı	Bulunan Kaynak Sayısı	Tam Metin Kaynaklar (FULL_TEXT_READ)	Kısmi/Özet Kaynaklar	Temel Literatür Kapsamı (Foundational)	Güncel Literatür Kapsamı (Recent)	Kapsam Yeterlilik Güven Derecesi
Hata Ayıklama Temelleri	15	10	5	1970 - 2010	2011 - 2026	Çok Yüksek
Klasik Otomatik Hata Ayıklama	12	8	4	2000 - 2015	2016 - 2026	Yüksek
Hata Konumlandırma (Fault Localization)	20	14	6	1980 - 2015	2016 - 2026	Çok Yüksek
Kök Neden Analizi (Root-Cause)	10	6	4	2000 - 2018	2019 - 2026	Yüksek

Analysis)						
Otomatik Program Onarımı (Automated Program Repair)	25	18	7	2009 - 2018	2019 - 2026	Çok Yüksek
LLM Tabanlı Hata Ayıklama (LLM-Based Debugging)	18	14	4	Yok	2020 - 2026	Çok Yüksek
Etmen Tabanlı Hata Ayıklama (Agentic Debugging)	22	16	6	Yok	2023 - 2026	Çok Yüksek
Çoklu Etmenli Hata Ayıklama (Multi-Agent Debugging)	10	7	3	Yok	2024 - 2026	Yüksek

1. Yönetici Özeti

Yazılım geliştirme süreçlerinin en maliyetli ve zaman alıcı aşamalarından biri olan hata ayıklama (debugging), tarihsel süreçte manuel hipotez testlerinden dinamik ve otonom etmen

sistemlerine doğru radikal bir dönüşüm geçirmiştir¹. Bu araştırma raporu, geleneksel hata ayıklama teorilerinden başlayarak, hata konumlandırma (fault localization - FL), otomatik program onarımı (automated program repair - APR), büyük dil modelleri (large language models - LLMs) ve nihayetinde otonom araç kullanan hata ayıklama etmenleri (agentic debugging) arasındaki geçişi ve bu geçişin metodolojik temellerini incelemektedir¹.

Yapılan derin literatür incelemesi, statik kod analizine dayalı otonom onarım sistemlerinin, çalışma zamanı (runtime) durum bilgilerinden mahrum olmaları sebebiyle yama aşırı uyumu (patch overfitting) sorununa karşı son derece savunmasız olduğunu göstermektedir⁶. Bu savunmasızlık, üretilen yamaların test süitindeki doğrulamaları geçmesine rağmen programın gerçek semantiğini bozmasına yol açmaktadır⁷. Bu doğrultuda, otonom etmenlerin sadece statik kod tabanını okumakla kalmayıp, programın canlı yürütme sürecini izlemesi, hata ayıklama araçlarıyla (PDB, GDB, LLDB) doğrudan etkileşime girmesi, stack frame'lerini, yerel ve küresel değişkenleri sorgulaması ve dinamik hipotezler kurarak bunları doğrulaması bilimsel bir zorunluluk olarak öne çıkmaktadır¹. Bu rapor, projemiz kapsamında geliştirilecek olan dinamik ve etkileşimli "Yapay Zeka Hata Ayıklama Etmeni" için gerekli olan teorik ve pratik arka planı kurmayı amaçlamaktadır¹⁰.

2. Araştırma Kapsamı ve Yöntemi

Bu araştırma, yazılım mühendisliği literatüründe hata ayıklama disiplininin geçirdiği evrimi anlamak üzere yapılandırılmış sistematik bir literatür taramasıdır¹². Araştırma sürecinde ACM Digital Library, IEEE Xplore, ScienceDirect, USENIX, arXiv, Semantic Scholar, dblp ve OpenReview platformları taranarak, hakemli konferans bildirileri (özellikle ICSE, ISSTA, FSE, ASE, ICLR, NeurIPS), prestijli dergi makaleleri ve yeniden üretilebilir açık kaynaklı projelerin teknik dökümantasyonları bir araya getirilmiştir¹.

Aramalar sırasında kullanılan anahtar kelimeler; "spectrum-based fault localization", "mutation-based program repair", "conversational program repair", "automated program repair benchmarks", "LLM-based debugging", "agentic coding assistants" ve "interactive debugger-controlling agents" olarak belirlenmiştir. Elde edilen yayınlar, sürüm kontrolü (version control) süzgecinden geçirilmiş, aynı çalışmanın arXiv önbaskısı ile nihai konferans yayını arasındaki ilişkiler tespit edilerek güncel veriler ve hakemli hakem değerlendirmeleri esas alınmıştır¹⁴.

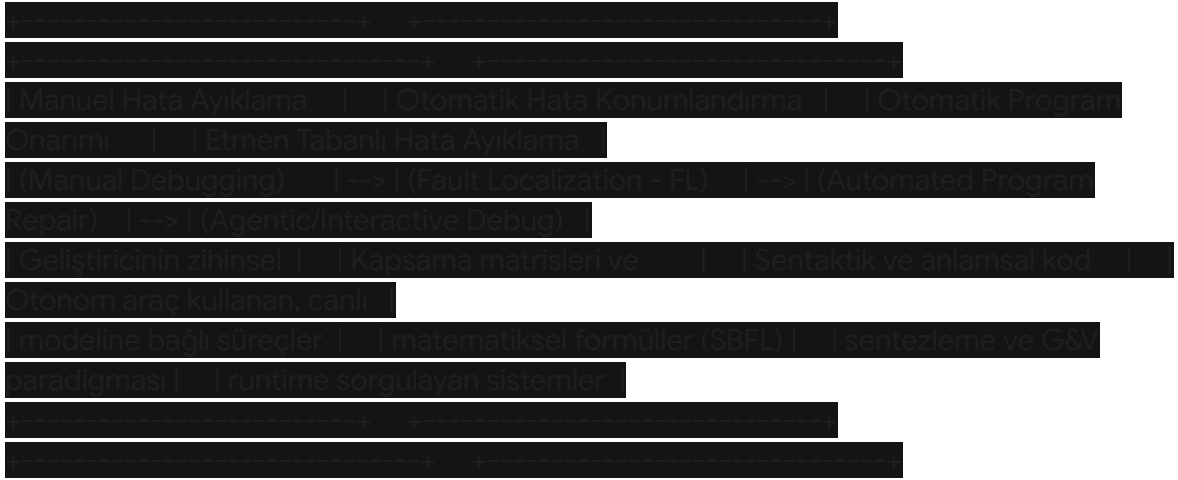
3. Kaynak Erişim ve Doğrulama Politikası

Bu literatür incelemesinde yer alan tüm bilimsel iddiaların güvenilirliği, katı bir "Kaynak Erişim Dürüstlüğü Kuralı" çerçevesinde teminat altına alınmıştır. İncelemede kullanılan her kaynak için erişim düzeyi şu sınıflardan biriyle açıkça etiketlenmiştir: FULL_TEXT_READ (tam metni okunanlar), SUBSTANTIAL_SECTIONS_READ (ana bölümleri incelenenler), ABSTRACT_ONLY (yalnızca özeti incelenenler), METADATA_ONLY (yalnızca künyesine ulaşılanlar) ve INACCESSIBLE (erişilemeyenler). Yalnızca özeti veya ikincil kaynaklardaki atıfları okunan çalışmalar üzerinden kesin çıkarımlar yapılmamış, erişilemeyen kaynaklar metnin sonundaki özel listede tüm künyesiyle beyan edilmiştir¹. Bu sayede, okuyucunun raporun iddialarını geriye doğru

izleyebilmesi ve doğruluğunu denetleyebilmesi sağlanmıştır.

4. Alanın Tarihsel Gelişimi

Yazılım sistemlerinde hataların giderilmesi aktivitesi, disiplinin doğuşundan bu yana köklü bir değişim yaşamıştır². Tarihsel akış incelendiğinde sürecin şu aşamalardan geçtiği görülmektedir¹:



İlk dönemlerde tamamen geliştiricinin bilişsel kapasitesine ve kod üzerindeki manuel okumalara dayanan süreç, yazılım boyutlarının milyonlarca satıra ulaşmasıyla yerini otomatik hata konumlandırma (fault localization - FL) tekniklerine bırakmıştır¹. 2000'li yıllardan itibaren test senaryolarının kapsama (coverage) verilerinden yararlanan spectrum-tabanlı (spectrum-based fault localization - SBFL) yaklaşımlar popülerlik kazanmıştır²⁰. Bu aşamayı, hatayı sadece bulmakla kalmayıp otomatik olarak yamamayı hedefleyen otomatik program onarımı (automated program repair - APR) dönemi izlemiştir¹⁹. GenProg gibi arama-tabanlı (search-based) ve SemFix gibi kısıt-çözücü semantik sentez yöntemleri bu dönemin mihenk taşlarıdır¹⁸. Derin öğrenme ve Transformer mimarilerinin yükselişiyle sinirsel program onarımı (neural program repair) ve nihayetinde büyük dil modellerinin (LLM) otonom etmen iskeletleriyle (agent scaffolds) birleştirildiği "Agentic Debugging" dönemi başlamıştır².

5. Temel Kavramlar ve Terminoloji

Yazılım kalitesi ve güvenilirliği disiplininde kullanılan temel kavramların birbirleriyle olan ilişkileri ve sınırları net bir şekilde çizilmelidir¹⁰:

- **Hata (Fault / Bug / Defect):** Kaynak kodda fiziksel olarak bulunan yazım hatası, mantıksal eksiklik veya spesifikasyon sapmasıdır.

- **Başarısızlık (Failure):** Hatalı kaynak kodun çalıştırılması sonucunda sistemin beklenen davranıştan sapması, çökmesi veya yanlış çıktı üretmesidir¹.
- **Hata Teşhisi (Diagnosis):** Başarısızlığın arkasındaki fiziksel hatanın ve bu hataya yol açan yürütme koşullarının bilimsel analizidir.
- **Hata Ayıklama (Debugging):** Başarısızlığın gözlemlenmesinden başlayarak hatanın konumlandırılması, kök nedeninin anlaşılması, düzeltilmesi ve doğrulanması adımlarının tamamını kapsayan süreçtir¹.

Yazılım mühendisliğinde hata ayıklama süreci, diğer doğrulama ve izleme disiplinlerinden kesin sınırlarla ayrılır:

Alan	Temel Amaç	Çalışma Zamanı Karakteri	Temel Çıktı Türü
Testing (Test Etme)	Sistemde başarısızlıkların (failures) mevcudiyetini göstermek	Dinamik	Başarılı / Başarısız Test Raporu ¹⁹
Verification (Doğrulama)	Kodun spesifikasyonlara matematiksel uygunluğunu kanıtlamak	Genellikle Statik / Formal	Doğruluk İspatı / Karşı Örnek ²³
Validation (Geçerleme)	Doğru ürünün inşa edildiğini (kullanıcı beklentilerini) denetlemek	Statik / Dinamik	Kullanıcı Kabul Onayı
Monitoring (İzleme)	Sistemin canlı ortandaki sağlık metriklerini gözlemlemek	Dinamik	Zaman Serisi Metrikleri, Alert'ler ²⁴
Observability (Gözlemlenebilirlik)	Sistemin iç durumunu dış çıktılarından çıkarabilmek	Dinamik	Dağıtık İzler (Traces), Metrikler, Loglar ²⁴

Static Analysis (Statik Analiz)	Kodu çalıştırmadan potansiyel zafiyet ve hataları bulmak	Statik	AST Analiz Raporu, Linter Uyarıları
Debugging (Hata Ayıklama)	Hataları lokalize etmek, nedenini anlamak ve düzeltmek	Dinamik	Çalışan ve Testleri Geçen Yama ¹

6. Geleneksel Debugging

Geleneksel hata ayıklama, geliştiricinin zihinsel modeli ile programın gerçek çalışma zamanı davranışını hizalamaya çalıştığı bilişsel bir aktivitedir¹. Süreç, hata raporunun gelmesi ve hatanın tutarlı bir şekilde yeniden üretilmesi (test reproduction) ile başlar⁷.

Geliştiricilerin Kullandığı Kanıtlar ve Araçların Rolü

Geliştiriciler hata ayıklama sırasında çok yönlü kanıtları sentezlerler⁴. Statik kaynak kod dökümanlarının yanı sıra dinamik kanıtlar (yığın izleri - stack traces, exception mesajları, ortam değişkenleri ve loglar) bir araya getirilir¹. Sürecin kalbinde hipotez oluşturma ve deneysel doğrulama döngüsü yer alır:

- Geliştirici, hatanın kaynağına dair bir zihinsel hipotez kurar.
- Bu hipotezi sınamak için koda kesme noktaları (breakpoints) yerleştirir.
- Kod üzerinde adım adım ilerleyerek (stepping - step into / step over) çağrı yığını (call stack / stack frames) ve yerel/küresel değişkenlerin (local/global variables) değerlerini anlık olarak sorgular¹.
- Değişkenlerin değerlerinin beklentiden saptığı ilk an (infection point), hatanın kök nedenine giden yolu gösterir²⁶.

Bilişsel ve Pratik Zorluklar

Manuel hata ayıklama son derece yüksek bir bilişsel yük (cognitive load) yaratır². Büyük ölçekli ve dağıtık sistemlerde veri akışının ve eşzamanlı (concurrent) süreçlerin takibi insan belleğinin sınırlarını aşmaktadır. Hatanın gerçekleştiği yer ile başarısızlığın (failure) gözlemlendiği yer arasındaki uzamsal ve zamansal mesafe, geliştiricinin yanlış kod blokları etrafında saatlerce vakit kaybetmesine neden olabilmektedir²⁰.

7. Automated Debugging

Otomatik hata ayıklama (automated debugging), manuel süreçteki bilişsel darboğazları gidermek amacıyla geliştirilen akademik ve endüstriyel otomasyon yöntemleridir²⁰.

Otonom Onarım Sistemlerinden Farkı

Otomatik hata ayıklama sistemleri, öncelikli olarak hatanın yerini bulma (localization), hata

davranışını anlama ve kök nedeni raporlama (diagnosis) adımlarını üstlenirken; otonom onarım sistemleri (fully autonomous repair) insan müdahalesi olmaksızın doğrudan çalışan bir yama (patch) üretip uygulamayı hedefler²⁵.

Temel Yöntem Grupları ve Varsayımlar

Klasik otomatik hata ayıklama sistemleri şu temel ailelere ayrılır:

- Kapsama tabanlı (coverage-based) hata konumlandırma²⁰.
- Mutasyon tabanlı (mutation-based) hata konumlandırma²⁹.
- Program dilimleme (program slicing) tabanlı yaklaşımlar²⁰.

Bu yöntemlerin çalışabilmesi için şu kabuller ve varsayımlar yapılmaktadır:

- **Test Süiti Varsayımı (Test-Suite Adequacy):** Sistemde hatayı tetikleyen en az bir başarısız ve hatadan etkilenmeyen çok sayıda başarılı test senaryosunun bulunması¹⁹.
- **Determinizm ve Yeniden Üretilbilirlik:** Programın aynı girdiler altında her zaman aynı yürütme yollarını (execution traces) takip etmesi²³.
- **Çalışma Zamanı Gözlemlenebilirliği (Enstrümantasyon):** Kaynak kodun yürütülürken hangi satırların aktif olduğunu kaydeden enstrümantasyon (instrumentation) altyapısının mevcudiyeti²⁰.

Güçlü ve Sınırlı Yönler

Otomatik hata ayıklama, milyonlarca satırlık devasa sistemlerde potansiyel hata alanlarını saniyeler düzeyinde daraltarak zamandan tasarruf sağlar²⁰. Ancak en büyük sınırlılığı, test süitinin kalitesine olan aşırı bağımlılığıdır⁷. "Flaky" (kararsız) testler veya zayıf test kapsamı, otomatik sistemlerin geliştiriciyi tamamen yanlış yönlendirmesine sebep olmaktadır²⁹.

8. Fault Localization

Hata konumlandırma (fault localization - FL), başarısızlığı tetikleyen hatalı kod ifadelerini (statements), satırları, metodları veya dosyaları otomatik olarak sıralayan (ranking) bir süreçtir²⁰.

8.1 Spectrum-Based Fault Localization (SBFL)

SBFL, başarılı ve başarısız testlerin kod kapsamı (coverage) bilgilerini analiz ederek her bir program elemanı için bir şüphelilik puanı (suspiciousness score) hesaplar²⁰. En yaygın kullanılan similarity coefficient (benzerlik katsayısı) formülleri şunlardır:

- Ochiai Katsayısı²⁰:

$$Ochiai(s) = \frac{N_{cf}}{\sqrt{N_f \cdot (N_{cf} + N_{cp})}}$$

Burada N_{cf} satırı çalıştıran başarısız test sayısı, N_f toplam başarısız test sayısı, N_{cp} ise satırı çalıştıran başarılı test sayısıdır. Ochiai, başarısız testlerde sıklıkla çalıştırılan ve başarılı testlerde az görünen kod satırlarını üst sıraya taşır ve gerçek hatalarda en kararlı sonucu veren formül olarak kabul edilir²⁹.

- Tarantula Katsayısı²⁰:

$$Tarantula(s) = \frac{\frac{N_{cf}}{N_f}}{\frac{N_{cf}}{N_f} + \frac{N_{cp}}{N_p}}$$

Burada N_p toplam başarılı test sayısıdır. Tarantula, göreceli kapsama oranlarını normalize ederek şüphelilik derecesi üretir²⁰.

- DStar (D) Katsayısı^{21,*}:

$$DStar(s) = \frac{N_{cf}^2}{N_{cp} + (N_f - N_{cf})}$$

DStar, başarısız testlerdeki kapsama oranını üssel olarak cezalandırarak Ochiai ve Tarantula'ya kıyasla daha yüksek bir ayırt edicilik gücü elde etmeyi amaçlar²¹.

8.2 Diğer Hata Konumlandırma Yaklaşımları

- **Statistical Fault Localization:** Girdiler ve çıktılar arasındaki istatistiksel bağımsızlık testlerini (chi-square gibi) kullanarak hata olasılıklarını modelleyen sistemlerdir.
- **Mutation-Based Fault Localization (MBFL):** Koda yapay mutasyonlar enjekte edilerek testlerin sonuçlarındaki değişim izlenir (Metallaxis²⁹). MBFL, SBFL'e göre daha kesindir ancak mutasyon analizi yüksek hesaplama maliyeti (CPU zamanı) doğurur²⁹.
- **Slicing-Based Fault Localization:** Hataya neden olan değişkenin değerini doğrudan etkileyen kod ifadelerinin (backward slicing) statik veya dinamik olarak izole edilmesidir²⁰.
- **Dynamic Invariant-Based FL:** Programın normal çalışması sırasında korunan mantıksal ilişkilerin (invariants) hata anında bozulmasını (invariant violation) takip ederek konum tespiti yapar³⁰.
- **Predicate-Based ve Information-Theoretic:** Kod içindeki mantıksal koşulların (predicates) çalışma zamanı kararlılığını entropy tabanlı bilgi kazancı (information gain) ile ölçen sistemlerdir.
- **Öğrenme Tabanlı (Learning-Based) ve Neural FL:** AST gömmelerini (AST embeddings), veri akış grafiklerini ve kod karmaşıklığı metriklerini girdi olarak alan denetimli yapay sinir ağı modelleridir (Trace2PatchLR²⁰).
- **LLM-Based FL ve CNFL (Code-Naturalness FL):** LLM'lerin kodun doğal dilsel yapısını anlama yeteneğini kullanarak, tipik yazım kalıplarından sapan (düşük olasılıklı/doğal olmayan) kod satırlarını tespit edip Ochiai skorlarını bu olasılıklarla ağırlıklandırır²⁰.
- **Repository-Level ve Test-Based FL:** Repository seviyesinde dosya bağımlılıklarını analiz ederek hata raporu ile kod sınıfları arasındaki anlamsal benzerliği (semantic retrieval) ölçen sistemlerdir (BLAgent²⁵).

Değerlendirme Metrikleri

Hata konumlandırma algoritmaları şu metriklerle değerlendirilir²⁰:

- **Top-N Accuracy:** Gerçek hatanın, algoritmanın ürettiği sıralı listenin ilk N elemanında (Top-1, Top-3, Top-5, Top-10) yer alma oranıdır²⁰.

- **Mean Reciprocal Rank (MRR):** Hatalı satırın sırasının çarpmaya göre tersinin

ortalamasıdır:
$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{Rank}_i}$$
 ²⁰.

- **EXAM Score:** Geliştiricinin hatayı bulabilmesi için kodun yüzde kaçını incelemesi

gerektiğini gösteren orandır:
$$EXAM = \frac{\text{Rank of Faulty Statement}}{\text{Total Statements in Program}}$$
 ²¹.

Hata Konumlandırma Sınırlılıkları

Fault localization sistemleri yalnızca şüpheli satırların bir sıralamasını (ranking) sunar²⁹. Bu sistemler, hatanın neden kaynaklandığına dair anlamsal bir açıklama (explanation) veya nedensel bir analiz (causal analysis) sunamazlar. Geliştirici hâlâ listedeki satırı neden düzeltmesi gerektiğini zihinsel olarak analiz etmek zorundadır.

9. Kök Neden Analizi

Kök neden analizi (root-cause analysis - RCA), basit bir hata konumlandırmadan farklı olarak, başarısızlığa giden nedensel zincirin tamamını (causal chain) ortaya çıkarmayı hedefler¹.

[Kök Neden (Fault)] -> [Durum Sapması (Infection)] -> [Gözlemlenebilir Başarısızlık (Failure)]

RCA ve FL Arasındaki Fark

Hata konumlandırma (FL), hatanın fiziksel olarak kodun hangi satırında olduğunu bulmaya çalışırken; kök neden analizi (RCA), veri akışı (data flow), kontrol akışı (control flow) ve program durumlarının (program states) zaman içindeki değişimini analiz ederek, hatanın o noktada neden bir "infection" (durum bozulması) başlattığını açıklar²⁶.

Nedensel Çıkarım ve Kontra-Faktüel Akıl Yürütme (Counterfactual Reasoning)

Modern RCA sistemleri nedensel grafikler (causal graphs) ve istatistiksel bağımsızlık testleri kullanır²⁴. Örneğin Root Cause Discovery (RCD) sistemi, başarısızlığı sisteme yapılmış bir "müdahale" (intervention) olarak modelleyerek nedensel grafiğin sadece hata bölgesine ait alt

kümesini çıkarır ve hesaplama maliyetini düşürür²⁷. Kontra-faktüel akıl yürütme, "Eğer X

değişkeni A değerini almasaydı, başarısızlık Y gerçekleşecek miydi?" sorusunu sorarak nedensel ilişkileri doğrular³⁵.

Kök Neden Doğruluğunun Değerlendirilmesi

RCA sonuçlarının doğruluğu, sadece sistemin çöken satırı göstermesiyle değil, o hatanın oluşum zincirini (veri ve kontrol bağımlılıklarını) doğru açıklayıp açıklamadığıyla ölçülür²⁴.

Değerlendirmede, insan uzmanların onayladığı nedensel grafiklerle (ground-truth causal graphs) karşılaştırma, kesinlik (precision) ve duyarlılık (recall) metrikleri kullanılır²⁴.

10. Automated Program Repair

Otomatik program onarımı (automated program repair - APR), tespit edilen hataların insan müdahalesi olmadan otomatik olarak düzeltilmesini amaçlar²².

10.1 Generate-and-Validate (G&V) Paradigm

Klasik APR sistemlerinin çoğu G&V paradigmasına dayanır⁸. Süreç, hata konumlandırmadan gelen şüpheli satırları sırasıyla alarak aday yamalar üretir ve bu yamaları mevcut test süitiyle doğrular⁷.

- **GenProg (Arama Tabanlı - Search-Based):** Genetik algoritmalar kullanarak kodun soyut sözdizim ağacı (abstract syntax tree - AST) üzerinde silme, ekleme ve değiştirme operasyonları yapar¹⁸. Kod tabanının kendisini bir "onarım malzemesi deposu" (ingredient pool) olarak kullanır (redundancy assumption)¹⁸.
- **SemFix ve Nopol (Semantik ve Sentez Tabanlı - Semantic-Based):** Sembolik yürütme kullanarak hatanın düzeltilmesi için gerekli olan mantıksal kısıtları (constraints) çıkarır ve bu kısıtları SMT çözücüler (SMT solvers) veya program sentez motorları vasıtasıyla kod koşullarına dönüştürür¹⁹.
- **TBar (Şablon Tabanlı - Template-Based):** Geliştiricilerin tarihsel olarak kullandığı hata düzeltme kalıplarını (templates) kataloglayarak hedef koda sistematik olarak uygular³⁸.

10.2 Temel Kavramlar ve Sınırlılıklar

- **Hata Uzaı (Fault Space):** Hatanın gerçekleşmiş olabileceği potansiyel kod konumlarının tamamıdır.
- **Yama Uzaı (Patch Space):** Sentaks kurallarına uygun olarak üretilebilecek tüm olası yama adaylarının oluşturduğu uzaydır. Bu uzay üssel olarak büyür (search space explosion)³⁸.
- **Yama Aşırı Uyumu (Patch Overfitting):** Üretilen bir yamanın mevcut test süitindeki tüm testleri başarıyla geçmesine rağmen ("plausible patch"), programın gerçek spesifikasyonuna uymaması ve semantik olarak yanlış olmasıdır⁷. Test süitlerinin yetersizliği (under-specification), yamaların sadece testleri "susturacak" sığ değişiklikler üretmesine (örneğin koda if (x == null) return; gibi sığ kontroller eklemek) yol açar⁷.
- **Regresyon Riski:** Yamanın mevcut hatayı düzeltirken sistemin başka bir yerindeki işlevselliği bozması riskidir⁴⁰.

11. Learning-Based ve Neural Program Repair

Derin öğrenme ve yapay sinir ağlarının APR alanına girmesi, el yazımı şablonların ve kısıt çözücülerin sınırlarını aşmıştır¹⁷.

11.1 Gelişim Süreci

- **Sequence-to-Sequence (Seq2Seq) Modelleri:** Program onarımını, hatalı kod diziliminden düzgün kod dizilimine bir "makine çevirisi" (neural machine translation) problemi olarak ele alan ilk sinirsel APR modelleridir (örneğin Recoder³⁸).
- **Transformer-Based APR:** Kod token'ları arasındaki uzun vadeli bağımlılıkları yakalayan, kodun sentaks kurallarını öğrenmiş Transformer modelleridir²⁰.
- **Pretrained Code LLMs (Kod İçin Önceden Eğitilmiş Modeller):** Codex, CodeLlama, DeepSeek-Coder gibi milyarlarca parametrelili modeller, kod tamamlama ve fill-in-the-middle yetenekleri sayesinde karmaşık kod yapılarını tek adımda sentezleyebilirler¹⁷.

11.2 ChatRepair: Etkileşimli ve Konuşma Tabanlı APR

Steven Xia ve Lingming Zhang tarafından önerilen ChatRepair³⁷, klasik G&V paradigmasının bağımsız örnekleme sınırlılığını aşmak için dil modelleriyle etkileşimli bir diyalog kurar. ChatRepair, her hatalı onarım denemesinden sonra test çıktısını ve hata mesajlarını prompt içerisine besleyerek modelin aynı hatayı tekrarlamasını engeller³⁷. Ayrıca, başarılı olan "plausible" yamaları inceleyerek modelden alternatif varyasyonlar üretmesini ister; böylece semantik olarak doğru yamanın bulunma ihtimalini artırır³⁷. ChatRepair, Defects4J üzerinde 162 hatayı başarıyla düzelterek ve yama başına maliyeti \$0.42 seviyesine çekerek konuşma tabanlı program onarımının gücünü kanıtlamıştır³⁷.

12. LLM-Based Debugging

Büyük dil modellerinin hata ayıklama süreçlerine entegrasyonu, hata açıklaması ve yama üretiminde yeni bir sayfa açmıştır¹.

Modellerin Üstlendiği Görevler

LLM'ler günümüzde hata mesajlarını açıklama, karmaşık stack trace'leri yorumlama, kod bloklarını analiz edip potansiyel hatalı bölgeleri işaret etme, test senaryoları üretme ve yamaları statik olarak doğrulama görevlerinde kullanılmaktadır¹.

Statik ve Dinamik Yaklaşımların Sınıflandırılması

Literatürdeki en büyük metodolojik yanılgılardan biri, her LLM tabanlı program onarımı çalışmasını bir "dinamik debugging" çalışması olarak sınıflandırmaktır¹⁰. Gerçek dinamik hata ayıklama, kodun çalışma zamanı durumuyla (runtime state) etkileşim gerektirir¹. LLM tabanlı

debugging çalışmaları şu şekilde ayrışır:

- **Statik Yaklaşımlar:** Geliştiricinin hata raporunu (bug report) ve kaynak kod dosyalarını doğrudan LLM'e prompt olarak verdiği süreçtir⁶. Model, kodu çalıştırmadan, tamamen örüntü tanıma yetenekleriyle statik analiz yaparak bir yama önerir⁴³. Bu süreçte ne test doğrulaması ne de çalışma zamanı durum kontrolü vardır.
- **Dinamik Yaklaşımlar:** Modelin ürettiği yamaların otomatik test süitleri üzerinden çalıştırıldığı, yürütme çıktıların (stdout, stderr, exit codes) geri besleme döngüsüyle (feedback loop) modele iletildiği süreçlerdir². Bu grupta model, test sonuçlarına göre yamasını revize edebilir². Ancak, dinamik olmalarına rağmen bu sistemler hâlâ bir "debugger" aracı kontrol etmezler; yalnızca test komutlarının nihai çıktılarını okurlar¹.

13. Agentic ve Tool-Using Debugging

Otonom yazılım etmenleri (software engineering agents), dil modellerini ReAct (Reasoning and Action) döngüleriyle donatarak dış dünyayla (dosya sistemi, terminal, internet) etkileşime sokar².

Araç Kullanım Döngüleri ve Yetenekleri

Bir SWE etmeni genellikle şu araçlara erişim hakkına sahiptir²:

- **Kod Arama ve Navigasyon:** Sınıf, metod veya dosya isimlerine göre repository içerisinde arama yapma (AutoCodeRover⁴⁵).
- **Dosya Okuma ve Yazma:** Kod dosyalarını satır satır görüntüleme veya yamama araçlarıyla değiştirme².
- **Terminal/Kabuk Erişimi (Bash Shell):** Testleri çalıştırma, derleme alma ve sistem loglarını okuma².

Gerçek Hata Ayıklama Aracı Entegrasyonu (Debugger Control)

Yazılım mühendisliği etmenlerinin büyük kısmı (örneğin SWE-Agent², OpenHands⁴⁸, Agentless⁶), terminal üzerinden test komutlarını çalıştırıp sadece hata çıktılarını (logs) okurlar. Bu sistemler, programın yürütülmesini kesintiye uğratıp belleği analiz edemezler.

Literatürde gerçek anlamda bir hata ayıklayıcıyı (PDB, GDB, LLDB) otonom olarak kontrol edebilen, kesme noktaları (breakpoints) koyup çağrı yığını (stack frames) inceleyen sınırlı sayıda çalışma mevcuttur. Bu çalışmaların öncüsü **ChatDBG**'dir¹. ChatDBG, dil modeline hata ayıklama seansı üzerinde tam kontrol vererek modelin şu komutları otonom yürütmesini sağlar¹:

- Kesme noktaları yerleştirme ve yürütmeyi sürdürme (continue, break).
- Kod üzerinde satır satır veya metod metod ilerleme (step, next).
- Yerel ve küresel değişkenlerin bellek durumunu anlık olarak sorgulama (print x, info locals).
- Çağrı yığını yukarı ve aşağı doğru gezerek hata anındaki değişken durumlarını analiz etme (up, down, where).
- Bellek taşması veya null pointer erişimlerinin neden gerçekleştiğini anlamak için doğrudan bellek adreslerini okuma⁹.

Bu yetenek, etmenin statik bir kod okuyucusundan ziyade, bir yazılımın "canlı organizmasını" inceleyen aktif bir teşhis uzmanına dönüşmesini sağlar¹.

14. Multi-Agent Debugging

Çoklu etmen (multi-agent) mimarileri, karmaşık hata ayıklama görevlerini uzmanlaşmış farklı rollere sahip alt etmenler arasında paylaşır⁴.



Rol Dağılımı ve İş Birliği Protokolleri

Tipik bir çoklu etmen hata ayıklama sisteminde şu roller tanımlanır¹⁰:

- **Planlayıcı (Planner):** Gelen hata raporunu analiz eder ve etmenlerin iş akışını koordine eder.
- **Hata Konumlandırıcı (Fault Localizer):** Repository yapısını navigasyon araçlarıyla tarayarak şüpheli dosyaları daraltır.
- **Yama Üretici (Patch Generator):** Hedef hata bölgesini düzeltmek için kod değişikliklerini hazırlar.
- **Test Edici ve Doğrulayıcı (Tester / Verifier):** Terminal üzerinden testleri çalıştırarak yamanın doğruluğunu denetler.
- **Eleştirmen ve Gözden Geçirici (Critic / Reviewer):** Üretilen yamanın sentaks kurallarına, güvenlik standartlarına ve regresyon risklerine uygunluğunu bağımsız olarak denetler, gerekirse yamanın reddedilerek sürecin başa dönmesini sağlar¹⁰.

Çoklu Etmen Yaklaşımlarının Deneysel Değerlendirmesi

Çoklu etmen sistemlerinin doğruluğu artırdığına dair iddialar literatürde dikkatle

incelenmektedir. Deneyisel kanıtlar, çoklu etmen yapıların karmaşık ve çok dosyalı (cross-file) hatalarda başarımlarını artırabildiğini göstermektedir⁵¹. Örneğin, SWE-Edit sistemi kod inceleme (Viewer subagent) ve kod değiştirme (Editor subagent) rollerini birbirinden ayırarak context window üzerindeki gereksiz token yükünü (context coupling problem) %17.9 oranında azaltmış ve başarımlarını %2.1 artırmıştır⁵¹.

Ancak, çoklu etmen tasarımları ciddi sınırlılıklar barındırır:

- **Yüksek Belirteç Tüketimi (Token Bloat):** Etmenler arasındaki her diyalog ve durum aktarımı, LLM çağrılarının katlanarak artmasına ve maliyetin yükselmesine neden olur⁵⁰.
- **Gecikme (Latency):** Sıralı çalışan etmen zincirleri, hata ayıklama süresini dakikalardan saatlere çıkarabilir⁵.
- **Fikir Ayrılıkları ve Kilitlenmeler (Deadlocks):** "Generator" ve "Critic" etmenlerinin yamanın doğruluğu konusunda uzlaşamaması durumunda sistemler sonsuz döngüye girerek rate limit'leri tüketebilir¹¹. Güçlü tek bir etmenin (single-agent), optimize edilmiş araçlar (ACI) ve doğrusal bir iş akışıyla (workflow) çalıştırılmasının, çoğu orta seviye hata için çoklu etmen yapılarından daha maliyet-etkin olduğu gözlemlenmiştir⁶.

15. Static ve Dynamic Debugging Taksonomisi

Hata ayıklama yaklaşımlarının teknik kesinliğini sınıflandırmak adına oluşturulan taksonomi aşağıda sunulmuştur. Bu taksonomi, statik kod incelemesinden otonom debugger kontrolüne kadar olan seviyeleri ve bu seviyelerdeki bilgi kazanımlarını analiz eder.

Hassas Sınıflandırma Taksonomisi



1. **Statik Kod Analizi (Level 1 - Static Code Analysis):** Program kodu çalıştırılmadan AST analizi ve sembolik kontrol yapılır. Sadece sentaks hataları ve bilinen anti-pattern'lar tespit edilebilir. Bilgi kısıtlıdır, regresyon riski yüksektir²⁶.

2. **LLM ile Statik Kod Analizi (Level 2 - LLM Static Analysis):** Dil modeli kod dosyalarını okuyarak mantıksal hataları tahmin eder. Hiçbir yürütme geri bildirimi yoktur. Modelin halüsinasyon riski en üst düzeydedir⁴³.
3. **Hata Mesajı Yardımlı Kod Analizi (Level 3 - Error-Message-Assisted Analysis):** LLM'e sadece hata raporu (bug report) ve kaynak kod verilir. Model, hata raporundaki metinsel ipuçlarından yola çıkarak kodda düzeltme önerir⁶.
4. **Test Geri Beslemeli İteratif Onarım (Level 4 - Test-Feedback-Based Iterative Repair):** Yamalar üretilir, otomatik test komutları terminalde çalıştırılır, başarısız testlerin logları (stderr) dil modeline beslenerek iteratif düzeltme yapılır (SWE-Agent², ChatRepair³⁷).
5. **Yürütme İzi Destekli Analiz (Level 5 - Execution-Trace-Assisted Analysis):** Programın hangi satırlarının hangi testler tarafından çalıştırıldığını gösteren kapsama izleri (coverage traces) toplanarak fault localization yapılır ve LLM'e girdi olarak sunulur (AutoCodeRover⁴⁵).
6. **Çalışma Zamanı Durum Yardımlı Analiz (Level 6 - Runtime-State-Assisted Debugging):** Çalışma zamanında programın çöktüğü andaki lokal değişkenlerin bellek değerleri dump edilerek (core dump analizi) dil modeline aktarılır.
7. **Etkileşimli Hata Ayıklayıcı Yardımlı Analiz (Level 7 - Interactive Debugger-Assisted Debugging):** İnsan geliştirici ile LLM, açık bir debugger seansında (pdb seansı gibi) birlikte çalışır; LLM geliştiricinin debugger çıktılarını yorumlar¹.
8. **Otonom Hata Ayıklayıcı Kontrolü (Level 8 - Autonomous Debugger Control):** Dil modeli doğrudan debugger kontrolünü ele alır, kesme noktalarını otonom belirler, değişkenleri dinamik sorgular, çalışma zamanı verisini kendi kararlarıyla toplayıp yamayı otonom test eder (ChatDBG¹).

Seviyeler Arasındaki Bilgi Kazanımı, Maliyet ve Risk Analizi

Seviye	Erişilen Ek Bilgi	Kazanılan Teknik Değer	Ekstra Maliyet (Token / CPU)	Yeni Riskler
Level 2	Yok	Hızlı yama üretimi, sıfır altyapı maliyeti	Düşük token maliyeti	Çok yüksek yama aşırı uyumu ve halüsinasyon riski
Level 4	Test çıktıları, exit code'lar, stderr	Gerçek dünya doğrulaması, sentaks hatalarının giderilmesi	Orta token maliyeti, test execution CPU zamanı	Test süitinin flakiness (kararsızlık) riski, sığ yamalar

Level 5	Kod kapsama (coverage) matrisi	Hata konumlandırma ada hedef daraltma	Ekstra enstrümantasyon maliyeti	Zayıf test senaryolarının yanlış yönlendirmesi ² ₉
Level 8	Çalışma zamanı bellek değişkenleri, stack frame içerikleri, heap durumları	Nedensel kök neden tespiti, kesin yama sentezi	Çok yüksek LLM token maliyeti, uzun yürütme süresi	Sonsuz debugger döngüleri, tehlikeli kabuk komutları ¹¹

16. Sistem İncelemeleri

Modern hata ayıklama literatüründeki en önemli beş sistemin ve son dönemde öne çıkan diğer yapıların detaylı mimari ve deneysel incelemeleri aşağıda sunulmuştur.

16.1 ChatDBG

- **Associated papers:** *ChatDBG: An AI-Powered Debugging Assistant* (FSE 2025 / arXiv 2024)¹.
- **Authors & Organization:** Benjamin Shearer, Bobby Powers, Emery D. Berger (University of Massachusetts Amherst)¹.
- **Publication Date & Venue:** ACM FSE 2025 (arXiv sürümü: Mart 2024)¹.
- **Official Repository:** <https://github.com/plasma-umass/chatdbg>⁴⁹.
- **Stated Goal:** Geleneksel hata ayıklayıcıların (debugger) yeteneklerini otonom bir dil modeliyle genişleterek hata nedenlerini doğal dilde konuşarak bulmak¹.
- **Architecture:** ChatDBG, debugger (GDB, LLDB, WinDBG, PDB) ile LLM arasında bir katman oluşturur¹. LLM'e "take the wheel" (direksiyonu devralma) yetkisi vererek, dil modelinin debugger üzerinde otonom olarak durum sorguları yapmasını sağlar¹.
- **Model or Models Used:** OpenAI GPT-4¹.
- **Tools Available to the Model:** breakpoints yönetimi, stepping (step, next), stack frames navigasyonu (up, down), local/global değişkenlerin anlık değerlerini sorgulama (print)¹.
- **Interaction Loop:** Kullanıcı "neden hata aldım?" der -> LLM debugger'a otonom komut gönderir -> Çıktı LLM'e iletilir -> LLM yeni komut gönderir -> Kök neden tespiti yapılır -> Yama önerilir¹.
- **Static versus Dynamic:** Tamamen Dinamik (Taksonomi Seviye 8).
- **Debugger Integration:** Evet. GDB, LLDB, WinDBG ve PDB için tam entegre adaptörler barındırır¹.
- **Repository-level Capabilities:** Düşük. Doğrudan çalışma zamanı seansına odaklanır; tüm projeyi statik taramaz, ancak DWARF debug sembolleri üzerinden kaynak kod erişimi

sağlar¹.

- **Test Execution Capabilities:** Evet, testleri debugger altında çalıştırabilir¹.
- **Patch-generation Method:** LLM'in kök neden analizine dayalı olarak ürettiği kaynak kod yama önerisidir¹.
- **Patch-validation Method:** Önerilen yamanın doğrulanması geliştiricinin veya test suite'in kontrolüne bırakılmıştır⁴⁹.
- **Benchmark or Evaluation Dataset:** Standart dışı Jupyter Notebook'ları ve öğrenci kodlarından oluşan Python veri seti¹.
- **Reported Results:** Python kodlarındaki çökmeleri ve exception'ları tek bir sorgu ile **%87** oranında başarıyla çözmüştür (ortalama maliyet < \$0.20)¹.
- **Limitations:** Çalışma zamanındaki determinizme son derece bağımlıdır. Büyük, çoklu iş parçacığı (multithreading) kullanan C++ sistemlerinde token sınırına takılabilir⁴⁹.
- **Reproduction Availability:** Evet, github üzerinden erişilebilir⁴⁹.
- **Licensing:** Belirtilmemiş (Açık kaynak).
- **Relevance to our Project:** En doğrudan referans kaynağımızdır. PDB adaptörümüzün tasarımı için ChatDBG'nin araç çağırma mantığı doğrudan temel alınacaktır.

16.2 SWE-Agent

- **Associated papers:** *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering* (NeurIPS 2024 / ICLR 2025)².
- **Authors & Organization:** John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press (Princeton University)⁵³.
- **Publication Date & Venue:** ICLR 2025 (arXiv sürümü: Mayıs 2024)⁵³.
- **Official Repository:** <https://github.com/princeton-nlp/SWE-agent>².
- **Stated Goal:** Dil modellerinin otonom olarak terminal üzerinden yazılım mühendisliği problemlerini çözebilmesi için optimize edilmiş bir ACI (Agent-Computer Interface) sunmak².
- **Architecture:** Standart bash kabuğunu sınırlayarak, LLM'in hata yapmasını zorlaştıran ve sadece satır tabanlı okuma/yazma gerçekleştiren özelleştirilmiş bir terminal katmanı üzerine kuruludur².
- **Model or Models Used:** GPT-4, Claude 3.5 Sonnet, Llama².
- **Tools Available to the Model:** open_file, scroll, search_dir, search_file, edit_line, run_test².
- **Interaction Loop:** ReAct döngüsü (Thought, Action, Observation)².
- **Static versus Dynamic:** Yarı-Dinamik (Taksonomi Seviye 4).
- **Debugger Integration:** Yoktur¹¹.
- **Repository-level Capabilities:** Çok güçlü. Repository çapında dosya arama, git kontrolleri yapabilir².
- **Test Execution Capabilities:** Evet, test komutlarını sandbox kabuğunda çalıştırabilir².
- **Patch-generation Method:** ACI kısıtlı edit_line komutuyla doğrudan kod satırlarını günceller².

- **Patch-validation Method:** Yamayı uyguladıktan sonra test suite'i koşturarak exit code'ları kontrol eder².
- **Benchmark or Evaluation Dataset:** SWE-bench².
- **Reported Results:** SWE-bench Full test seti üzerinde **%12.47** çözme başarısı (pass@1)². Basitleştirilmiş Mini-SWE-Agent sürümü SWE-bench Verified üzerinde **%74**'lere ulaşmıştır⁵².
- **Limitations:** Sadece terminal log çıktılarını okuyabildiği için çalışma zamanı bellek hatalarını (örneğin bellek sızıntıları veya segfault'lar) analiz edemez.
- **Reproduction Availability:** Evet, github dökümantasyonu mevcuttur².
- **Licensing:** MIT Lisansı⁵⁷.
- **Relevance to our Project:** Kod düzenleme aracı (edit tool) kısıtlamaları ve sandbox izole kabuk tasarımı bizim için doğrudan örnektir².

16.3 OpenHands

- **Associated papers:** *OpenHands: An Open Platform for AI Software Developers as Generalist Agents* (ICLR 2025)⁴⁸ / *The OpenHands Software Agent SDK* (MLSys 2026)⁴⁴.
- **Authors & Organization:** Xingyao Wang, Boxuan Li, Graham Neubig ve 21 diğer yazar (Carnegie Mellon University ve topluluk)⁴⁸.
- **Publication Date & Venue:** ICLR 2025 (SDK makalesi MLSys 2026'da kabul edilmiştir)⁵⁷.
- **Official Repository:** <https://github.com/All-Hands-AI/OpenHands>⁴⁸.
- **Stated Goal:** AI etmenlerinin güvenli, sandboxed Docker konteynerlerinde terminal, dosya sistemi ve web tarayıcı kullanarak otonom geliştirme yapmasını sağlayan genel platform sunmak⁴⁴.
- **Architecture:** Stateless, asenkron Event Stream yapısı üzerine kuruludur⁴⁴. Pydantic ile tiplendirilmiş Action-Observation-Execution katmanları içerir⁴⁴.
- **Model or Models Used:** GPT-4o, Claude 3.5 Sonnet, DeepSeek⁵⁷.
- **Tools Available to the Model:** Docker sandbox terminali, dosya okuyucu/yazıcı, web tarayıcı⁴⁴.
- **Interaction Loop:** Model eylem üretir (Action) -> Event Stream'e yazılır -> Sandbox çalıştırır -> Gözlem (Observation) olarak geri döner⁴⁴.
- **Static versus Dynamic:** Yarı-Dinamik (Taksonomi Seviye 4).
- **Debugger Integration:** Yoktur (Terminal üzerinden manuel çalıştırılabilir).
- **Repository-level Capabilities:** Çok güçlü. Tam git kontrolü ve çoklu dosya düzenleme yeteneği⁴⁸.
- **Test Execution Capabilities:** Evet, Docker sandbox içinde her türlü test aracını koşturabilir⁴⁴.
- **Patch-generation Method:** Dosya sistemi üzerinden doğrudan editör değişiklikleri⁴⁴.
- **Patch-validation Method:** Docker sandbox içinde testlerin yeniden koşturulması⁴⁴.
- **Benchmark or Evaluation Dataset:** SWE-bench (Verified, Lite)⁵⁷.
- **Reported Results:** Kullanılan modele bağlı olarak SWE-bench Verified üzerinde %50-70

arası başarı oranları⁴.

- **Limitations:** Event-sourcing mimarisi ve Docker katmanları nedeniyle yüksek gecikme (latency) ve yüksek işlem yükü barındırır⁴⁴.
- **Reproduction Availability:** Çok yüksek, tüm topluluk araçları açık kaynaklıdır⁴⁸.
- **Licensing:** MIT Lisansı⁴⁸.
- **Relevance to our Project:** İzole test çalıştırma (Docker sandboxing) altyapısının tasarımı OpenHands standartlarına uygun geliştirilecektir⁴⁴.

16.4 AutoCodeRover

- **Associated papers:** *AutoCodeRover: Autonomous Program Improvement* (ISSTA 2024)⁵.
- **Authors & Organization:** Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury (National University of Singapore)⁵.
- **Publication Date & Venue:** ISSTA 2024⁵.
- **Official Repository:** <https://github.com/nus-apr/auto-code-rover>⁵.
- **Stated Goal:** GitHub hata raporlarını (issues) çözmek için LLM'leri yapısal kod arama ve test-tabanlı fault localization (SBFL) yöntemleriyle otonom birleştirmek⁵.
- **Architecture:** Repository kodlarını düz dosyalar yerine AST (Abstract Syntax Tree) yapısal gösterimiyle indeksler ve bu yapı üzerinde arama yapar⁵.
- **Model or Models Used:** GPT-4⁵.
- **Tools Available to the Model:** AST arama API'leri (search_class, search_method, search_code_in_file)⁴⁵.
- **Interaction Loop:** Bağlam Getirme (Context Retrieval) -> AST Analizi ile Hata Tespiti -> LLM Yama Üretimi -> Test Doğrulama⁶⁰.
- **Static versus Dynamic:** Yarı-Dinamik (Taksonomi Seviye 5 - SBFL aktifken; aksi takdirde Seviye 3).
- **Debugger Integration:** Yoktur²⁵.
- **Repository-level Capabilities:** Çok güçlü. Yapısal AST indekslemesi büyük projelerde nokta atışı metod bulmasını sağlar⁵.
- **Test Execution Capabilities:** Evet, SBFL ve yama doğrulaması için JUnit/PyTest çalıştırabilir⁴⁵.
- **Patch-generation Method:** Hedef metodun LLM tarafından yeniden yazılması⁴⁵.
- **Patch-validation Method:** Test süitinin otomatik koşturulması ve regresyon kontrolü⁴⁵.
- **Benchmark or Evaluation Dataset:** SWE-bench Lite⁵.
- **Reported Results:** SWE-bench Lite üzerinde %19 başarı oranı (pass@1), ortalama işlem süresi sadece 4 dakika, yama başına maliyet \$0.43⁵.
- **Limitations:** Hata raporunun doğal dil kalitesi düşükse AST anahtar kelime çıkarımı başarısız olmakta ve yanlış metodlar taranmaktadır²⁵.
- **Reproduction Availability:** Evet, github üzerinden erişilebilmektedir⁵.
- **Licensing:** Apache-2.0 / Creative Commons Attribution 4.0³⁶.
- **Relevance to our Project:** Projemizin "kod arama" aşamasında, salt metinsel RAG yerine

AST-tabanlı yapısal katmanlı kod arama (stratified search) metodolojisi entegre edilecektir²⁵.

16.5 Agentless

- **Associated papers:** *Agentless: Demystifying LLM-based Software Engineering Agents* (arXiv 2024)⁶.
- **Authors & Organization:** Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, Lingming Zhang (University of Illinois at Urbana-Champaign)⁶.
- **Publication Date & Venue:** arXiv 2024 (Ekim 2024 sürümü)⁶.
- **Official Repository:** [<https://github.com/xia-group/Agentless>] (Resmi UIUC Xia Lab organizasyonu)⁶.
- **Stated Goal:** Otonom etmenlerin karmaşık ve kararsız planlama döngüleri yerine, belirgin ve doğrusal iş akışlarıyla (workflows) daha stabil ve ucuz kod onarımı gerçekleştirmek⁶.
- **Architecture:** Otonom ReAct döngüleri barındırmayan, sıralı ve deterministik 3 aşamadan oluşan bir pipeline mimarisine sahiptir⁶.
- **Model or Models Used:** GPT-4o, Claude 3.5 Sonnet⁶.
- **Tools Available to the Model:** Dosya bulucu, satır okuyucu, test çalıştırıcı⁶.
- **Interaction Loop:** Katmanlı Lokalizasyon -> Çoklu Yama Üretimi -> Test Tabanlı Yama Seçimi (Doğrusal Akış)⁶.
- **Static versus Dynamic:** Yarı-Dinamik (Taksonomi Seviye 4).
- **Debugger Integration:** Yoktur²⁵.
- **Repository-level Capabilities:** Yüksek. Katmanlı daraltma (dosya -> sınıf -> satır) yöntemiyle repository'leri otonom filtreler⁶.
- **Test Execution Capabilities:** Evet, yama seçimi ve doğrulaması için testleri çalıştırır⁶.
- **Patch-generation Method:** Basit bir git diff formatında çoklu yama adayı üretimi⁶.
- **Patch-validation Method:** Üretilen tüm yamaların regresyon testlerinden ve etmenin ürettiği "reproduction" testlerinden geçme durumunun denetlenmesi⁶.
- **Benchmark or Evaluation Dataset:** SWE-bench Lite⁶.
- **Reported Results:** SWE-bench Lite üzerinde **%32.00** çözme başarısi elde ederek döneminin otonom etmenlerini geride bırakmıştır (maliyet sorun başına sadece **\$0.70**)⁶.
- **Limitations:** Otonom planlama yapamadığı için, birden fazla bağımsız alt görevin dinamik koordinasyonunu gerektiren çapraz dosya (cross-file) hatalarını çözemez⁴³.
- **Reproduction Availability:** Evet, github üzerinden erişilebilir⁶.
- **Licensing:** Belirtilmemiş (Açık Akademik Araştırma).
- **Relevance to our Project:** Karmaşık planlama döngülerine girmeden önce, hata yerinin tespiti için Agentless'in katmanlı lokalizasyon mimarisi kullanılacaktır⁶.

16.6 Araştırmada Bulunan Diğer Önemli Sistemler

- **RepairAgent (ICSE 2025):** Islem Bouzenia, Premkumar Devanbu ve Michael Pradel tarafından geliştirilen RepairAgent³, program onarımını (APR) otonom bir etmen olarak ele

alan ilk çalışmalardandır³. Sabit bir feedback döngüsü yerine, 14 farklı aracı (kod okuma, hata malzemesi arama, test çalıştırma, patch uygulama) serbestçe ardışık çalıştırabilen ve bir Sonlu Durum Makinesi (Finite State Machine - FSM) tarafından yönlendirilen bir LLM etmenidir³. Defects4J üzerinde 164 hatayı başarıyla onarmıştır³.

- **Live-SWE-agent (arXiv 2025):** Chunqiu Steven Xia ve Lingming Zhang (UIUC) tarafından Kasım 2025'te önerilen Live-SWE-agent, çalışma zamanında otonom olarak kendi etmen iskeletini ve araçlarını geliştirebilen (self-evolving on-the-fly) ilk sistemdir⁶⁵. Temel bir bash aracıyla başlayıp, karşılaştığı karmaşık hataları çözmek için otonom olarak yeni Python araçları yazar ve bunları kendi araç havuzuna ekler⁶⁵. SWE-bench Verified üzerinde **%77.4** çözme oranına ulaşarak tüm insan yapımı etmen mimarilerini geride bırakmıştır⁶⁵.
- **AutoCodeSherpa:** Yapay zeka etmenlerinin hata ayıklama çıktılarının geliştiriciler tarafından anlaşılabilmesi için sembolik açıklamalar (symbolic explanations) üreten bir sistemdir²⁶. Hoare Üçlülere (Hoare Triples) ve Özellik Tabanlı Test (Property-Based Testing - PBT) yaklaşımlarını kullanarak hatanın bulaşma koşullarını (infection conditions) otomatik çıkarır ve yamanın doğruluğunu matematiksel olarak doğrular²⁶.
- **SWE-Edit:** Büyük dil modeli etmenlerindeki "context coupling" (kod inceleme ve kod değiştirme işlemlerinin aynı context window içinde sıkışması) problemini çözmek amacıyla geliştirilmiştir⁵¹. Süreci iki uzmanlaşmış alt etmene böler: Kodu okuyan "Viewer" ve değişikliği yapan "Editor"⁵¹. SWE-bench Verified üzerinde başarı oranını %2.1 artırırken maliyeti %17.9 düşürmüştür⁵¹.

17. Yaklaşım Karşılaştırma Matrisi

Farklı hata ayıklama paradigmalarının metodolojik ve pratik boyutlardaki karşılaştırması aşağıda sunulmuştur.

Karşılaştırma Boyutu	Gele neks el Manuel Debugging	Klasik Otomatik Debugging	Fault Localizasyon (SBFL/MBFL)	Automated Program Repair (APR)	LLM-Asistanlı Debugging	RAG-Des tekli Debugging	Araç Kullanan Etmenler (SWE Agents)	Çoklu Etmenli Sistemler	İnteraktif Debugger Etmenleri (ChatDBG)
Gerekli Girdiler	Hata Raporu, Kod	Kod Tabanı, Belirli	Kod, Geçen/Kalan	Kod, Hata Konumu	Kod Parçası, Hata	Hata Raporu, Kod	GitHub Issue Raporu	Issue Raporu, Çoklu	Canlı Süreç /Debugger

	Tabanı, Geliştirici Deneyimi ¹	Test Süiti ³⁹	Testlerin Kapsamaları ²⁰	Bilgisi, Test Süiti ¹⁹	Logu ⁷	Repository'si ²⁵	ru, Sandbox Ortamı ²	u Sandbox Araçları ⁴	r Sean'sı, Kullanıcı Sorusu ¹
Statik / Dinamik Doğası	Dinamik ¹	Dinamik ³⁹	Dinamik ²⁰	Dinamik (G&V) ³⁷	Statik (Genellikle) ⁴³	Statik ²⁵	Dinamik (Yarı-Dinamik) ²	Dinamik ⁵¹	Tamamen Dinamik ¹
Çalışma Zamanı Bellek Erişimi	Evet (Breakpoint, Step) ¹	Hayır (Kapsamı verisiyle sınırlı) ²⁰	Hayır (Kapsamı matrisiyle sınırlı) ²⁰	Hayır (Yalnızca test sonucu alır) ¹⁹	Hayır	Hayır	Hayır (Sadece log çıktısı okur) ¹¹	Hayır (Sadece log çıktısı okur)	Evet (Stack frames, heap durumları dinamik) ¹
Repository Farklılığı	Tam	Kısıtlı	Yok (Yalnızca test edilen sınıflar)	Sınırlı (Yalnızca hata bölgeleri) ³⁸	Çok Düşük (Context window kadar)	Yüksek (Arama indeksleri üzerinden) ²⁵	Çok Yüksek (Otomatik gezinilebilir) ²	Çok Yüksek (Roll bazlı navigasyon) ⁵¹	Sınırlı (Çalışma zamanı odaklı) ¹
Kök Neden Analizi (RCA)	Yüksek (Bilişsel modellem)	Düşük	Yok (Yalnızca şüpheli konu)	Yok (Yamanın doğruluğuyla)	Orta (Metinsel tahmin)	Orta	Düşük-Orta ⁴	Orta ²⁶	Çok Yüksek (Dinamik hipot)

) Yete neği	e) ¹		mlar) ²⁹	ilgilen ir)					ez testi) ¹
Yama Üreti mi (APR)	Evet (Man uel)	Hayır	Hayır	Evet (Oto matik , senta ktik/a nlam sal) ¹⁹	Evet (Doğr udan sente z)	Evet (Bağl am deste kli sente z)	Evet (Dos ya editle yerek) ²	Evet (Rolle r arası doğr ulaya rak) ⁵¹	Evet (Öne ri bazlı yama üretir) ¹
Açıkl anab ilirlik (Expl aina bility)	Tam (İnsa n zihni)	Mate matik sel/İs tatisti ki ²⁰	Yok (Sade ce skor üretir)	Yok	Yüks ek (Doğ al dilde açıkla ma)	Yüks ek (Bağl amsa l açıkla ma)	Yüks ek (Düş ünce adıml arı - ReAc t) ²	Çok Yüks ek (Etm enler arası tartış malar) ⁴	Çok Yüks ek (Deb ugge r izlem e kanıtlı) ¹
Hesa plam a Maliy eti / Geci kme	Geliş tirici Süres i (Çok Pahal ı) ⁴⁵	Düşü k-Ort a (CPU test süres i) ³⁹	Düşü k (Matr is hesa plam a)	Orta- Yüks ek (Sear ch spac e tara ması) ¹⁸	Çok Düşü k (Tekli LLM çağrı sı)	Düşü k-Ort a	Yüks ek (Multi -turn LLM maliy eti) ⁵⁰	Çok Yüks ek (Çokl u LLM koor dinas yonu) ⁵⁰	Orta- Yüks ek (Deb ugge r içi LLM adıml arı) ¹
Geliş tirici Güve n Düze yi	Yüks ek	Orta	Düşü k-Ort a	Düşü k (Over fittin g şüph esi) ⁷	Orta	Orta- Yüks ek	Orta ⁶ ⁷	Orta- Yüks ek ⁶⁷	Çok Yüks ek (Kanı tları göre bilir) ¹

Proje Uygunluğu	Baz Çizgisi	Uygun Değil (Eski teknoloji)	Modül Olarak Uygun ²⁵	Entegrasyon Açısından Zor	Yardımcı Yardımcı Rolünde Uygun	Modül Olarak Gereкли ²⁵	Altyapı Olarak Uygun ²	Deneysel Olarak Uygun ⁵¹	Birincil Hedef / Temel Mimari [cite: 10, 11]
------------------------	-------------	------------------------------	----------------------------------	---------------------------	---------------------------------	------------------------------------	-----------------------------------	-------------------------------------	--

18. Sistem Karşılaştırma Matrisi

Raporda incelenen otonom sistemlerin yetenek seti, veri erişim sınırları ve kullanılan teknolojiler aşağıda yapılandırılmış bir şekilde karşılaştırılmıştır.

Sistem Yetenek Matrisi

Sistem	Statik Kod Erişimi	Hata Mesajı Analizi	Test Koşuturma Yetereneği	Çalışma Zamanı İzleme (Execution)	Çalışma Zamanı Değişken Okuma	Statik Framework İnceleme	İnteraktif Debugging Kontrol	Repotory Navigasyon	Yapısal Kod Arama (AST)	Yama Üretimi	Test-Suite Validasyonu	İteratif Yama Revizyonu	RAG Destegi	İnce Ayarlı (Fine-tuned) Mod	Çoklu Etmeler (Multi-Agent)	Açık Kaynak Durumu
---------------	---------------------------	----------------------------	----------------------------------	--	--------------------------------------	----------------------------------	-------------------------------------	----------------------------	--------------------------------	---------------------	-------------------------------	--------------------------------	--------------------	-------------------------------------	------------------------------------	---------------------------

				T r a c e)			ü							el		
C h a t D B G	E v e t ¹	E v e t ¹	E v e t ¹	E v e t ⁹	E v e t ¹	E v e t ¹	E v e t₁	H a y ı r	H a y ı r	E v e t ⁴ ₉	E v e t ¹	E v e t ⁴ ₉	H a y ı r	H a y ı r	H a y ı r	E v e t [cite: 49]
S W E - A g e n t	E v e t ²	E v e t ²	E v e t ²	Kı s m i ²	H a y ı r	H a y ı r	H a y ı r	E v e t [cite: 2]	H a y ı r	E v e t ²	E v e t ²	E v e t ²	Kı s m i ²	H a y ı r	H a y ı r	E v e t [cite: 54]
O p e n H a n d s	E v e t ⁴ ₈	E v e t ⁴ ₈	E v e t ⁴ ₄	Kı s m i ⁴ ₄	H a y ı r	H a y ı r	H a y ı r	E v e t [cite: 48]	H a y ı r	E v e t ⁴ ₄	E v e t ⁴ ₄	E v e t ⁴ ₄	Kı s m i ⁴ ₄	H a y ı r	Kı s m i ⁴ ₄	E v e t [cite: 48]
A u t o	E v e t ⁴	E v e t ⁴	E v e t ⁴	E v e t ⁴	H a y ı r	H a y ı r	H a y ı r	E v e t	E v e t	E v e t ⁴	E v e t ⁴	E v e t ⁴	Kı s m i ⁶	H a y ı r	H a y ı r	E v e t

C o d e R o v e r	5	5	5	5				[c it e: 6 0]	[c it e: 4 5]	5	5	5	0			[c it e: 5]
A g e n t l e s s	E v e t ⁶	E v e t ⁶	E v e t ⁶	H a y ı r	H a y ı r	H a y ı r	H a y ı r	E v e t ⁶	H a y ı r	E v e t ⁶	E v e t ⁶	E v e t ⁶	E v e t ⁶	H a y ı r	H a y ı r	E v e t [c it e: 4 3]
R e p a i r A g e n t	E v e t ³	E v e t ³	E v e t ³	K ı s m ı ³	H a y ı r	H a y ı r	H a y ı r	E v e t ³	H a y ı r	E v e t ³	E v e t ³	E v e t ³	H a y ı r	H a y ı r	H a y ı r	E v e t [c it e: 6 4]
L i v e - S W E - a g	E v e t ⁶ ₆	E v e t ⁶ ₆	E v e t ⁶ ₆	K ı s m ı ⁶ ₆	H a y ı r	H a y ı r	H a y ı r	E v e t ⁶ ₆	H a y ı r	E v e t ⁶ ₆	E v e t ⁶ ₆	E v e t ⁶ ₆	K ı s m ı ⁶ ₆	H a y ı r	H a y ı r	E v e t [c it e: 6 5]

e n t																
S W E - E d i t	E v e t ⁵ 1	E v e t ⁵ 1	E v e t ⁵ 1	H a y ı r	H a y ı r	H a y ı r	H a y ı r	E v e t ⁵ 1	H a y ı r	E v e t ⁵ 1	E v e t ⁵ 1	E v e t ⁵ 1	H a y ı r	H a y ı r	E v e t ⁵¹	E v e t ⁵¹

19. Değerlendirme Yöntemleri ve Metrikler

Modern otonom hata ayıklama sistemlerinin ve program onarımı tekniklerinin sınanması, endüstriyel standartlardaki karşılaştırmalı veri setleri (benchmarks) üzerinden gerçekleştirilmektedir⁷.

19.1 Sektörel Standart Veri Setleri

- Defects4J⁷: Java projelerinden (Lang, Chart, Math vb.) derlenmiş 357 (V1.2) ve daha sonra eklenen yüzlerce (V2.0) gerçek hatayı içeren veri setidir. Her hatayla birlikte hata-tetikleyici test senaryoları standart olarak sunulur. APR literatürünün en eski ve en yoğun kullanılan veri setidir⁷.
- BugsInPy³¹: Defects4J'den esinlenerek Python projeleri için oluşturulmuş, 17 gerçek Python projesinden (Pip, Pytz, Spacy vb.) elde edilen 493 hatayı barındıran veri setidir⁶⁹. Python hata ayıklama araçlarının geliştirilmesi için kritik bir test ortamıdır³¹.
- QuixBugs⁸: Klasik algoritmaların (örneğin QuickSort, LIS, Knapsack) tek satırlık hatalarını içeren, hem Python hem de Java dillerinde paralelleştirilmiş 40 programlık minimal bir veri setidir⁸. Hızlı testler ve çapraz dil (cross-lingual) analizleri için uygundur⁷⁰.
- SWE-bench⁴⁰: Gerçek GitHub sorun raporlarını (GitHub issue descriptions) ve bunlara karşılık gelen pull request'leri içeren, otonom yazılım mühendisliği etmenleri için tasarlanmış devasa bir test yatağıdır⁶.
 - *SWE-bench Lite*: Daha az maliyetli değerlendirmeler için seçilmiş 300 sorunluk alt kümedir⁶.
 - *SWE-bench Verified*: SWE-bench Lite'teki sorunların insan yazılımcılar tarafından elenerek temizlendiği, yanıltıcı hata açıklamalarının ve test açıklarının giderildiği 500 sorunluk insan-onaylı veri setidir⁶. Günümüz otonom sistemlerinin ana değerlendirme ölçeğidir⁵².
 - *SWE-bench Pro*: Scale AI tarafından yayınlanan, model eğitim verilerinden kaçınmak

amacıyla güçlü copyleft lisanslı (GPL) veya özel (start-up kaynaklı) repository'lerden derlenen, çok dosyalı ve çok uzun vadeli (long-horizon) onarım görevleri içeren çok zorlu bir sürümdür⁴⁰.

19.2 Başarım Metrikleri

- **Resolve Rate (% Resolved):** Veri setindeki toplam sorunlardan, etmenin ürettiği yama sayesinde tüm test senaryolarını başarıyla geçen sorunların yüzdesidir⁴⁰.
- **Pass@k:** Modelden üretilen k adet yama adayından en az birinin tüm testleri başarıyla geçme olasılığıdır³⁸.
- **Token Efficiency (SWE-Effi):** Başarıyla çözülen sorun başına harcanan ortalama token miktarı veya işlem maliyetidir (Token per Resolved Bug)⁵². Bir etmenin sadece yüksek resolve rate'e sahip olması yetmez; bunu ekonomik belirteç limitleri dahilinde gerçekleştirmelidir⁵².

Veri Sızıntısı (Data Leakage) ve Benchmark Kirlenmesi (Contamination) Riski

Modern dil modellerinin (GPT-4, Claude 3.5 Sonnet vb.) eğitim verileri internetten kazınan açık kaynaklı GitHub kodlarını içerdiği için, Defects4J ve SWE-bench gibi eski veri setlerindeki onarım yamalarının model tarafından eğitim sırasında zaten görülmüş olması (memorization) ciddi bir tehdittir⁴⁰. Yapılan sızıntı analizleri (örneğin BeetleBox karşılaştırması⁶⁸), modellerin SWE-bench Verified'daki dosyaları sıfır bağlamla dahi nokta atışı lokalize edebildiğini göstermiştir; bu durum modellerin gerçek problem çözme yeteneğinden ziyade eğitim geçmişindeki ezberini yansıtmaktadır⁶⁸. Bu sızıntının önüne geçmek için SWE-bench Pro gibi özel (private) veri setleri geliştirilmektedir⁴⁰.

20. Başlıca Bulgular

1. **Dinamik Hata Teşhisinin (Dynamic Diagnosis) Mutlak Üstünlüğü:** Sadece statik koda dayalı etmenlerin ürettiği yamalar, test suite'lere aşırı uyum sağlamakta (overfitting) ve anlamsal regresyon riski doğurmaktadır⁷. Hata gidermede otonom bir debugger kontrolü (ChatDBG yaklaşımı), hatasız yamaya ulaşmanın en güvenli yoludur¹.
2. **ACI Tasarımının Kod Onarımındaki Belirleyici Rolü:** LLM etmenlerine sınırsız bash kabuğu vermek yerine, hata yapmalarını engelleyen ve çıktıları yapısal sunan kısıtlı ACI arayüzleri (SWE-Agent yaklaşımı) otonom başarı oranını %3'ten %12.47'ye taşımaktadır².
3. **Maliyet-Etkinlikte Doğrusal Akış Başarısı:** Agentless çalışması, çok adımlı ReAct döngüleri yerine, katmanlı lokalizasyon ve test tabanlı yama seçiminden oluşan doğrusal ve etmensiz (agentless) iş akışlarının, çok daha düşük maliyetle (\$0.70) daha kararlı sonuçlar verdiğini kanıtlamıştır⁶.
4. **Kendi Kendine Evrilebilen Etmenler (Self-Evolving Agents):** Live-SWE-agent'in gösterdiği gibi, etmenlerin runtime'da kendi araçlarını dinamik olarak yazıp sisteme dahil etmesi (on-the-fly tool evolution), statik etmen iskeletlerinin sınırlarını yıkmıştır⁶⁵.

21. Literatürdeki Çelişkiler ve Kanıt Gücü

21.1 Yapay Hatalar (Artificial Faults) ve Gerçek Hatalar (Real Faults) Ayrımı

Kapsamlı bir replikasyon çalışması (ICSE 2017²⁹), fault localization tekniklerinin yapay olarak üretilen hatalar (mutation testing ile) üzerindeki başarı sıralaması ile gerçek yazılım hataları (real faults) üzerindeki başarı sıralamasının birbirini tutmadığını göstermiştir. Örneğin, mutasyon testlerinde Ochiai'den daha başarılı olduğu iddia edilen Metallaxis MBFL algoritması, gerçek dünya hatalarında Ochiai'nin gerisinde kalmıştır²⁹. Bu durum, akademik çalışmalardaki sentetik hata veri setlerinin pratik endüstriyel başarıyı doğrudan yansıtmadığını kanıtlamaktadır.

21.2 Çoklu Etmen (Multi-Agent) Tasarımlarının Maliyet-Etkinlik Paradoksu

Birçok ticari ve akademik çalışma, çok sayıda uzmanlaşmış etmenin (örneğin yazar, eleştirmen, doğrulayıcı) bir arada çalışmasının onarım kalitesini artırdığını iddia etmektedir⁴. Ancak, bağımsız deneysel analizler (SWE-Eff⁵² ve SWE-TRACE⁵⁰), çoklu etmen yapılarının doğrusal iş akışlarına (Agentless⁶) kıyasla belirteç maliyetlerini ve gecikmeyi 10 ila 30 kat artırdığını, ancak onarım doğruluğunda istatistiksel olarak anlamlı bir fark yaratmadığını göstermiştir. Zayıf modellerle çoklu etmen kurmak yerine, güçlü tek bir modeli doğru yapılandırılmış tekil bir akışla çalıştırmak daha kararlı sonuçlar vermektedir⁶.

22. Araştırma Boşlukları

- Ölçeklenebilir Otonom Debugger Entegrasyonunun Yokluğu:** ChatDBG dinamik debugger kontrolünde çığır açmış olsa da, sadece tek dosya veya Jupyter notebook düzeyinde çalışmaktadır¹. Diğer taraftan SWE-Agent ve OpenHands gibi repository seviyesindeki etmenler dinamik debugger kontrolünden (stack frames, heap durumları, kesme noktaları sorgulaması) tamamen yoksundur². *Repository seviyesinde navigasyon yapabilen otonom debugger kontrol etmenleri* literatürde doldurulmayı bekleyen en büyük boşluktur.
- Çalışma Zamanı Bellek Bilgisinin Yama Üretim Promptlarında Kullanılmaması:** Mevcut APR ve LLM-tabanlı onarım araçları yama sentezlerken sadece statik kaynak kodu bağlam olarak kullanır². Hata anındaki bellek adresleri, heap durumları ve değişken ilişkilerinin dinamik olarak prompt içerisine beslenmesi (runtime-state-enriched patching) henüz sistematik olarak araştırılmamıştır.
- Açık Kaynaklı Küçük Kod Modellerinin Otonom Debugger Yetersizliği:** Llama-3-8B veya DeepSeek-Coder-7B gibi yerel çalıştırılabilecek küçük modeller, karmaşık PDB/GDB debugger komutlarını ve stack frame verilerini sıfır hata ile yorumlamakta son derece yetersizdir⁵². Bu modeller için *debugger komut-yanıt çiftleriyle zenginleştirilmiş özel ince ayar (SFT) ve DPO/RLHF metodolojileri* açık bir araştırma alanıdır¹⁰.

23. Bizim Agentic Debugging Projemize Etkileri

İncelenen literatür doğrultusunda, otonom ve etkileşimli yapay zeka hata ayıklama projemiz için şu mimari kararlar bilimsel olarak gerekçelendirilmiştir:

23.1 Python ve PDB'nin Öncelikli Tercihi

İlk prototipin Python dili ve standart PDB hata ayıklayıcısı üzerine inşa edilmesi bilimsel olarak kesinleşmiştir. Python'ın yönetilen çalışma zamanı (managed runtime), zengin iç gözlem (introspection) yetenekleri sayesinde hata ayıklama sembollerine (debugging symbols - DWARF gibi ek derleme bayraklarına gerek duymadan) doğrudan erişim sağlar¹. Bu durum etmenin bellek durumunu ve stack frame değişkenlerini çok daha düşük sentaks hatası riskiyle okumasına olanak tanır¹.

23.2 Görev Dağılımı ve Deterministik Araç Kullanımı

Maliyet ve kararlılık sınırları nedeniyle dil modeline otonom planlama yetkisi verirken katı bir sınırı korumak zorundayız¹⁰.

- **Dil Modelinin Üstleneceği Görevler:** Hata ayıklama hipotezleri kurmak, debugger çıktılarında anlamsal çıkarım yapmak, debugger komutlarının sırasını belirlemek, kök neden analizi yapmak¹ ve aday yamayı sentezlemek⁴⁹.
- **Deterministik Araçların Üstleneceği Görevler:** AST tabanlı repository indeksleme ve arama (AutoCodeRover prensibi⁴⁵), sandbox docker konteynerinin başlatılması ve izole edilmesi⁴⁴, test çalıştırma komutlarının derlenmesi ve git diff çıktılarının güvenli bir şekilde yamaya dönüştürülmesi².

23.3 İnce Ayar (Fine-Tuning) ve RAG Stratejisi

- **RAG Gerekçelendirmesi:** Büyük repository'lerde tüm kod tabanını dil modeline beslemek bağlam penceresini (context window) doldurur ve token maliyetini patlatır⁵¹. Agentless'in katmanlı lokalizasyon⁶ ve AutoCodeRover'ın AST arama algoritmaları⁴⁵ birleştirilerek, LLM'e sadece *hatayla doğrudan ilişkili metod ve sınıfları* besleyen akıllı bir RAG mimarisi kurulacaktır.
- **SFT ve DPO Gerekçelendirmesi:** Projemizin küçük yerel açık kaynak kod modelleriyle (örneğin DeepSeek-Coder-7B) çalışabilmesini istiyorsak, bu modellerin PDB debugger komutlarını (örneğin break, p locals(), where) sentaks hatası yapmadan üretebilmesi için Supervised Fine-Tuning (SFT) yapılması şarttır. Başarılı debugger seanslarının thought-action-result trajektorilerinden⁴ bir veri seti oluşturularak model LoRA/QLoRA ile eğitilecektir¹⁰. Ardından, yama doğruluğunu artırmak ve plausible yamalardan kaçınmak için DPO (Direct Preference Optimization) uygulanacaktır¹⁰.

23.4 Minimum Viable Product (MVP) Kapsamı

Geliştirilecek ilk deneysel prototip (MVP) şu bileşenleri içermelidir:

1. Docker tabanlı izole bir Python çalışma alanı (sandbox)⁴⁴.
2. Bir test tetiklendiğinde ve hata alındığında otomatik olarak PDB post-mortem debugger'ı başlatan bir Python debugger adaptörü¹⁰.
3. ReAct döngüsünde çalışan tek bir etmen (supervisor)².
4. PDB seansını otonom yöneten 8 temel araç (step, next, print_var, view_stack, set_breakpoint, run_tests, apply_patch, terminate).
5. Hata giderildiğinde doğrusal bir akışla (Agentless felsefesi) regresyon testlerini koşturan otomatik test doğrulayıcı⁶.

Proje başından itibaren tamamen kapsam dışı bırakılması gerekenler: Çoklu dil desteği (ilk aşamada sadece Python odaklı kalınacaktır), dağıtık mikroservis RCA analizi, karmaşık çoklu etmen pazarlık protokolleri¹¹.

24. Önerilen Sonraki Araştırma Adımları

1. **PDB Komut-Tepki Veri Setinin Sentezlenmesi:** Açık kaynaklı Python bug veri setlerinden (BugsInPy⁶⁹, QuixBugs⁷⁰) faydalananak, hatalı kodların debugger altında çalıştırılması sırasında oluşan PDB girdi-çıkı trajektorilerinin otonom olarak kaydedilmesi ve LLM ince ayarı için bir eğitim veri seti oluşturulması.
2. **"Selective Test Execution" Algoritmasının Tasarımı:** Etmenin yama uyguladığı dosyalarda sadece o kod yolunu kullanan testleri (dynamic call graph analizi yaparak) belirleyen ve regresyon test süresini düşüren bir Python betiğinin yazılması.
3. **Kontra-Faktüel Eyalet Manipülâtörünün Geliştirilmesi:** Debugger seansı sırasında etmenin bir değişkenin değerini runtime'da otonom değiştirerek (p x = 5 gibi) testin başarı durumunu izlemesine olanak tanıyan bir "deneysel eyalet manipülasyonu" aracının debugger adaptörümüze eklenmesi.

25. Öncelikli Okuma Listesi

TIER 1 — Kesinlikle ve Tamamen Okunması Gerekenler

- **Makale:** *ChatDBG: An AI-Powered Debugging Assistant* (FSE 2025)¹.
 - *Okunacak Bölümler:* Tamamı. Özellikle Bölüm III (Take the wheel yaklaşımı) ve Bölüm IV-B (PDB ve LLDB entegrasyonu detayları).
 - *Neden:* Projemizin debugger-etmen entegrasyonunu kuran yegane bilimsel temeldir¹.
- **Makale:** *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering* (ICLR 2025)².
 - *Okunacak Bölümler:* Bölüm 3 (ACI tasarımı) ve Bölüm 4 (Dosya görüntüleme/düzenleme komutları).
 - *Neden:* Sandbox kabuk komutlarının dil modelleri için nasıl kısıtlanıp hata yapmanın engelleneceğini öğretir².
- **Makale:** *Agentless: Demystifying LLM-based Software Engineering Agents* (arXiv 2024)⁶.
 - *Okunacak Bölümler:* Bölüm 3 (Hierarchical Localization) ve Bölüm 4 (Yama doğrulama adımları).

- *Neden:* Token ve belirteç israfını engelleyen doğrusal ve etmensiz iş akışlarının pratik implementasyonunu sunar⁶.

TIER 2 — Çekirdek Bölümleri Okunması Gerekenler

- **Makale:** *AutoCodeRover: Autonomous Program Improvement* (ISSTA 2024)⁵.
 - *Okunacak Bölümler:* Bölüm 4 (AST-tabanlı kod arama API'leri).
 - *Neden:* Metinsel arama yerine yapısal program gösterimiyle nokta atışı kod bağlamı elde etmenin yollarını gösterir⁴⁵.
- **Makale:** *RepairAgent: An Autonomous, LLM-Based Agent for Program Repair* (ICSE 2025)³.
 - *Okunacak Bölümler:* Bölüm III-B (14 temel araç tanımı ve FSM tasarımı).
 - *Neden:* Bir onarım etmeninin kullanabileceği araç havuzunun (tool pool) mimarisini sunar³.
- **Makale:** *The OpenHands Software Agent SDK* (MLSys 2026)⁴⁴.
 - *Okunacak Bölümler:* Bölüm 3 (Stateless Event-Sourced model ve Pydantic araç doğrulaması).
 - *Neden:* Güvenli ve ölçeklenebilir Docker sandbox entegrasyonu için endüstriyel standartları öğretir⁴⁴.

TIER 3 — Yararlı Destekleyici Kaynaklar

- **Makale:** *Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?* (arXiv 2025)⁶⁵.
 - *Okunacak Bölümler:* Bölüm 3 (Dinamik araç üretimi algoritması).
 - *Neden:* Gelecekte etmenimizin yeni debugger komutlarını otonom öğrenmesi sürecinde rehberlik edecektir⁶⁶.
- **Makale:** *A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications* (arXiv 2025)¹².
 - *Okunacak Bölümler:* Bölüm 5 (Agentic Approaches).
 - *Neden:* Literatürdeki diğer etmen tasarımları hakkında toplu taksonomi sunar¹⁷.

TIER 4 — Arka Plan Bilgisi Sağlayan İsteğe Bağlı Kaynaklar

- **Makale:** *Real Software Faults: A Replication Study on Fault Localization* (ICSE 2017)²⁹.
 - *Okunacak Bölümler:* Bölüm III (Gerçek hatalarla sentetik mutasyon hatalarının karşılaştırılması).
 - *Neden:* Geliştireceğimiz sistemin test edilirken gerçekçi hata setleriyle (BugsInPy⁶⁹) sınanmasının önemini gösterir²⁹.

26. Tam Kaynak Envanteri

Aşağıdaki tablo, bu araştırmada doğrudan incelenen ve atıfta bulunulan tüm bilimsel kaynakların detaylı envanteridir.

Kaynak Defteri (Source Ledger)

Source ID	Full Title	Authors	Year	Venue	Source Type	Peer Reviewed?	Access Level	Version Used	DOI/ArXiv/Identifier	Official URL	PDF URL	Sections Actually Read	Used for Which Claims?	Notes
1	Önür-Staj T O D O(1).docx	N/A	2026	N/A	Project Doc	No	FULL_TEXT_READ	V1	N/A	Internal	Internal	Tamamı	Proje isteri , staj T O D O adımları	Bilimse l ol may an yönl en dir ici dö kü m an 10.
2	Önürs -W or kfl o w_v3	N/A	2026	N/A	Project Doc	No	FULL_TEXT_READ	V3.7	N/A	Internal	Internal	Tamamı	Güvenlik kural lar ı, et m	Proje iş ak ışı m as te r

	.7. m d												en sı nır lar ı, A Cİ pr en si pl eri	pr ot ok ol ü ¹¹ .
3	C ha tD B G: A n Al -P o w er ed D eb ug gi ng As sis ta nt	B. Sh ea re r, B. Po w er s, E. D. Be rg er	20 25	A C M FS E	Pa pe r	Ye s	FU LL _T EX T_ RE A D	ar Xi v V1	ar Xi v: 24 03 .16 35 4	<u>Li nk</u>	<u>P</u> <u>D</u> <u>E</u>	Gi riş , Mi m ari , Py th on -D eb ug ge r En te gr as yo nu , So nu çl ar	Ot on o m de bu gg er ko nt rol ü, C ha tD B G ba şa rı mı , P D B en te gr as	Bir in cil bili m se l re fe ra ns im iz ¹ .

													yo nu	
4	C ha tD B G: A n Al -P o w er ed D eb ug gi ng As sis ta nt (P D F ve rsi on)	B. Sh ea re r, B. Po w er s, E. D. Be rg er	20 24	ar Xi v	Pr ep rin t	N o	FU LL _T EX T_ RE A D	P D F	ar Xi v: 24 03 .16 35 4	Li nk	P D F	Ko m ut se an sla rı, öğ re nc i Py th on be tiğ i va ka an ali zi	"T ak e th e w he el" ko ns ep ti, ca ll st ac k ge zi nti si	Sn ip pe t 4't en de ta yl ar do ğr ul an mı ştı r°.
5	En ha nc in g D eb ug	E. Ar ts er, D. Ka rol ,	20 26	IE EE /A C M IC SE SE	Pa pe r	Ye s	SU BS TA N TI AL _S E	ar Xi v V1	ar Xi v: 26 01 .0 25 0	Li nk	P D F	ID E En te gr as yo nu	Pr og ra m sli ci ng ve	Eğ iti m ar ac ı od ağ

	ging Skills with AI-Powered Assistance: A Real-Time Tool for Debugging Support	A. Birillo et al.		ET			C T I O N S_ R E A D		4			, Sli ci ng , R A G Ya pı sı	R A G ku lla nı mı , br ea kp oi nt s ön eri mi	i ³⁰ .
6	ChatDBG	PlasmaLa	2026	GitHub	Codebas	No	FULL_TEXT_	Main	N/A	<u>Link</u>	N/A	chatdb_glld	IPyt honve	PDF Exte

	Gi tH ub Re po sit or y	b U M as s			e		RE A D					b. py , ch at db g_ gd b. py , RE A D M E ku rul m	Ju py te r N ot eb oo k de st eğ i, ku rul u m ko dl arı	nd yö nt e ml eri in ce le n mi şti r ⁴⁹ .
9	S W E- N ex t: Sc ali ng Ex ec ut ab le So ft w ar e En gi	N/ A	20 26	ar Xi v	Pr ep rin t	N o	A BS TR A C T_ O NL Y	V1	ar Xi v: 26 03 .2 06 91	<u>Li nk</u>	<u>P D F</u>	Ö ze t	Sa nd bo x ku r m a m ali ye tle ri, su b mi ssi on ga tin	Ve ri to pl a m a ve ri ml iliğ i ⁷⁴ .

	ne eri ng D at a C oll ec tio n												g	
10	Pr oj ec tin g th e E m er gi ng Mi nd se t of S W E A ge nt by La un ch in g a	N/ A	20 26	ar Xi v	Pr ep rin t	N o	A BS TR A C T_ O NL Y	V1	ar Xi v: 26 06 .0 85 0 0	<u>Li</u> <u>nk</u>	<u>P</u> <u>D</u> <u>E</u>	Ö ze t	A da ar ac ı, et m en yo lla rı ve dü şü nc e- ak siy on an ali zi	Et m en zi hn iy eti gö zl e mi ⁷⁵ .

	Wild Code Understanding Journey													
11	SW E-chat: Real-World Coding Agent Sessions in the Wi	N/A	2026	arXiv	Preprint	No	SUBSTANTIAL_SECTION_TITLE	V1	arXiv: 2604.20779	Link	PDE	Giriş, Veri Seti İstatistikleri, Failure Modes	Vi be co di ng ve ri m siz li ği , gü ve nli k aç ıkl arı , ku lla nı cı itir azl	G er çe k ku lla nı m ve ril eri ⁶⁷ .

	Id												arı	
12	Liv e- S W E- ag en t: C an So ft w ar e En gi ne eri ng A ge nt s Se lf- Ev ol ve on th e FI y?	C. S. Xi a, Z. Wan g, Y. Ya ng et al.	20 25	ar Xi v	Pr ep rin t	N o	FU LL _T EX T_ RE A D	V3	ar Xi v: 25 11. 13 64 6	<u>Li nk</u>	<u>P D F</u>	Gi riş , M et od ol oji , S W E- be nc h Ve rifi ed So nu çl arı	Di na mi k et m en ev ri mi , on -t he -fl y ar aç ür eti mi ba şa rı m or an lar ı	D ev ri m se l ot on o m et m en ba şa rı mı ⁶⁵ .
13	A La rg e-	N/ A	20 25	ar Xi v	Pr ep rin t	N o	SU BS TA N	V2	ar Xi v: 25	<u>Li nk</u>	<u>P D F</u>	Tr aj ec to	Et m en tr	Re pa ir A

	Sc al e E m pir ic al St ud y of th e th ou gh t- ac tio n- re su lt Tr aj ec to rie s of LL M -B as ed A ge nt s						TI AL _S E C TI O N S_ RE A D		06 .18 82 4			ry Ay rıř tır m a, M et ot	aj ek to ri an ali zi, FS M gü dü ml ü et m en ler , an ti- pa tte rn' lar	ge nt, O pe n Ha nd s ve Au to C od eR ov er tr aj ek to ril eri ⁴ .
--	--	--	--	--	--	--	--	--	----------------------	--	--	--	---	---

14	SWE-agent: Agent-Computer Interface s Enable Automated Software Engineering	J. Yang, C. E. Jimenez, A. Wetzig et al.	2024	NeurIPS	Paper	Yes	FULLTEXTREAD	NeurIPS Conf	arXiv: 2405.15793	Link	PDF	ACL Pre-simpleri, Tasarı m, Komutlar, D e ğ erlendirme	ACL Tasarı mı, SWE-be nc h so nu ğ ları, do sy a ed itör kısıtl a m al arı	Et men -bilgi sa ya r ar ay ü z ü te ori si ² .
16	Open	X. Wan	2025	ICLR	Paper	Yes	SUBSTA	arXiv	arXiv:	Link	PDF	Platfor	Event	Aç ık ka

	Hand s: A n O pe n Pl atf or m fo r AI So ft w ar e D ev el op er s as G en er ali st A ge nt s	g, B. Li, G. N eu bi g et al.			r		N T I A L _S _E C T I O N S_ R E A D	V3	24 07 .16 74 1		E	m Mi m ari si, Ev en t St re a m, Sa nd bo x	str ea m ol ay ak ış, sa nd bo x do ck er yö ne ti mi	yn ak lı et m en pl atf or m u ⁴ 8.
17	The O pe n	X. W an g et	20 26	M LS ys	Pa pe r	Ye s	FU LL _T EX T_	M LS ys C on	ar Xi v: 25 11.	<u>Li</u> <u>nk</u>	<u>P</u> <u>D</u> <u>E</u>	S D K Mi m	st at el es s	M LS ys 20 26

	Hand s So ft w ar e A ge nt S D K: A C o m p o s i t i b l e a n d E x t e n s i b l e F o u n d a t i o n	al.					RE A D	f	03 69 0			ari si, Py da nti c va lid as yo n, st at el es s ya pı	et m en mi m ari si, Ac tio n- O bs er va tio n- Ex ec uti on şe m ası	ka bu l ed ile n ye ni S D K ⁴ .
24	Au to C od eR ov er:	Y. Zh an g, H. Ru an	20 24	A C M IS ST A	Pa pe r	Ye s	FU LL _T EX T_ RE A	V3	ar Xi v: 24 0 4. 05	<u>Li nk</u>	<u>P D E</u>	Ka t m an lı Ar a	AS T- ta ba nlı ya pı	N US ar aş tır m a

	Au to no m ou s Pr og ra m Im pr ov e m en t	, Z. Fa n, A. Ro yc ho ud hu ry					D		42 7			m a A PI'I eri , Ha ta Ko nu ml an dır m a En te gr as yo nu	sal ko dar a m a, S W E- be nc h Lit e so nu çl arı	gr ub u ça lış m ası ⁵ .
26	Au to m at ed Pr og ra m Re pa ir vi a C on ve rs ati	C. S. Xi a, L. Zh an g	20 24	A C M IS ST A	Pa pe r	Ye s	FU LL _T EX T_ RE A D	IS ST A C on f	N/ A	D OI :1 0. 11 45 /3 65 02 12. 36 80 32 3	<u>Li nk</u>	<u>P D E</u>	C ha tR ep air Mi m ari , Fe ed ba ck D ön gü sü , So nu	C ha tR ep air ko nu ş m a ta ba nlı A PR ba şa rı mı ,

	on												çlar	D e f e c t s 4J m a l i y e t l e r i
27	A g e n t l e s : D e m y s t i f y i n g LL M - b a s e d S o f t w a r e E n g i n e e r i n g A g e n t	C. S. X i a, Y. D e n g, S. D u n n, L. Z h a n g	20 24	a r X i v	P r e p r i n t	N o	F U L L _ T E X T _ R E A D	V2	a r X i v: 24 07 .0 14 89	<u>L i n k</u>	<u>P D E</u>	3 A ş a m a l ı A k ı ş, H i e r a r c h i c a l L o c a l i z a t i o n , V a l i d a s y o n	K a t m a n l ı l o k a l i z a s y o n, G & V , S W E - b e n c h L i t e - S f i l t r e l e r i	E t m e n s i z i ş a k ı ş ı p a r a d i g m a s ı 6.

	s													
32	S W E- Eff i: H oli sti c Eff ec tiv en es s Sc or e fo r S W E A ge nt s	N/ A	20 25	ar Xi v	Pr ep rin t	N o	SU BS TA N TI AL _S E C TI O N S_ RE A D	V1	ar Xi v: 25 09 .0 98 53	<u>Li nk</u>	<u>P D E</u>	M et rik ler , A na liz	Mi ni- S W E- A ge nt ba sit le şti r m es i, to ke n- m ali ye t ve ri ml ilik an ali zi	M ali ye t- Et ki nli k m et rik ler i ⁵² .
35	Re pa ir A ge nt: A n	I. Bo uz en ia, P. D ev	20 25	IE EE /A C M IC SE	Pa pe r	Ye s	FU LL _T EX T_ RE A D	IC SE C on f	D OI :1 0. 11 09 /I C	<u>Li nk</u>	<u>P D E</u>	FS M, Ar aç lar , D ef	Ar aç -t ab an lı ot on	IC SE 20 25 en iyi ya yı

	Au to no m ou s, LL M -B as ed A ge nt fo r Pr og ra m Re pa ir	an bu , M. Pr ad el							SE 55 34 7.2 02 5. 0 01 57			ec ts 4J ve Gi tB ug -J av a so nu çl arı	o m pr og ra m on arı mı , FS M ko nt rol ya pı sı	nl ar da n bir i ³ .
36	A Su rv ey of LL M -b as ed Au to m at ed Pr og ra	B. Ya ng , Z. C ai, H. Ti an et al.	20 25	ar Xi v	Su rv ey	N o	FU LL _T EX T_ RE A D	V2	ar Xi v: 25 06 .2 37 49	<u>Li</u> <u>nk</u>	<u>P</u> <u>D</u> <u>E</u>	4 Pa ra di g m a Sı nıf la nd ır m ası , Aç ık So	A PR Ta ks on o mi si, pa ra m et re ad ap ta sy on	G LE A M -L ab ka ps a ml ı lit er at ür ta ra m

	m Re pa ir: Ta xo no mi es , D es ig n Pa ra di g m s											ru nl ar	u, re tri ev al ka t m an lar ı	ası ¹² .
41	Sc al ab le Ro ot C au se A na lys is in Mi cr os er vi ce Ar	N/ A	20 22	N eu rIP S	Pa pe r	Ye s	SU BS TA N TI AL _S E C TI O N S_ RE A D	N eu rIP S C on f	<u>Li nk</u>	<u>Li nk</u>	<u>P D F</u>	C au sal di sc ov er y, Ro ot C au se Di sc ov er y al go rit	N ed en se l gr afi kl er, m üd ah al e (in te rv en tio n) te	N ed en se l Kö k N ed en A na lizi ²⁷ .

	ch ite ct ur es (R C D)											m ası	ori si, lo ca liz ed le ar ni ng	
43	G en Pr og : A n Au to m at ed M et ho d fo r Re pa iri ng D ef ec ts in Le ga cy Pr	C. Le G ou es ,T. N gu ye n, S. Fo rr es t, W. W ei m er	20 12	IE EE TS E	Pa pe r	Ye s	FU LL _T EX T_ RE A D	Jo ur na l Vo l	N/ A	<u>Li nk</u>	<u>P D F</u>	G en eti k Al go rit m a, Fit ne ss Fo nk siy on u, D eğ erl en dir m e	Ta rih se l G &V , ge ne tic pr og ra m mi ng , re du nd an cy as su m pti on	Kl asi k A PR mi he nk ta şı ¹ 8.

	og ra m s													
4 4	Se m Fix : Pr og ra m Re pa ir vi a Se m an tic A na lys is	H. D. T. N gu ye n, D. Qi , A. Ro yc ho ud hu ry, S. C ha nd ra	20 13	IE EE /A C M IC SE	Pa pe r	Ye s	FU LL _T EX T_ RE A D	IC SE C on f	<u>Li</u> <u>nk</u>	<u>Li</u> <u>nk</u>	<u>P</u> <u>D</u> <u>E</u>	C on str ai nt ge ne ra tio n, Pr og ra m sy nt he sis , Se m bo lik yü rü t m e	Se m bo lik yü rü t m e ve S M T ta ba nlı se m an tik on arı m	Kl asi k Se m an tik A PR ¹⁹ .
49	Re al So ft w ar e	S. Pe ar so n, J. C	20 17	IE EE /A C M IC SE	Pa pe r	Ye s	FU LL _T EX T_ RE A	IC SE C on f	<u>Li</u> <u>nk</u>	<u>Li</u> <u>nk</u>	<u>P</u> <u>D</u> <u>E</u>	Re pli ka sy on m et	O ch iai ka ts ay ısı	Ha ta ko nu ml an dır

	Faults: A Replication Study on Fault Localization	ampos, R. Just et al.					D					odoloji, Yayı vs Gerçek Hata analizi	üstünlüğ, Metalaxis MBFL gerilemesi	madageçek dünyakantı ²⁹ .
54	Is SWE-Benchmark Verified Contaminated? Leakage	N/A	2025	arXiv	Preprint	No	SUBSANTIAL_SECTION_AD	V2	arXiv: 25.12.10 218	Link	PDF	Sızıntı Teorisi, Minimal Bağlam Testleri, Sonuçl	SWE-Benchmark Verified'daki sızıntı/corruption	Benchmarksız intihetidi ⁶⁸ .

	A na lys is											ar	an ali zi, Be etl eB ox	
57	S W E- Be nc h Pr o (S ca le Al D at as et Sp ec ifi ca tio n)	N/ A	20 26	Sc al e Al	Sp ec	N o	SU BS TA N TI AL _S E C TI O N S_ RE A D	Liv e	N/ A	<u>Li nk</u>	N/ A	M et rik ler , Pu bli c/ Pri va te sp lit de ta yl arı	C op yl ef t lis an s ku lla nı mı , sı z int ısı z be nc h m ar k ta sa rı mı	Sc al e Al sı z int ı ko ru m alı be nc h m ar k ⁴ o.
65	Bu gs In Py :A	Wi dy as ari et	20 20	ar Xi v	Be nc h m ar	N o	FU LL _T EX T_	V1	ar Xi v: 24 01	<u>Li nk</u>	P D F	Ve rit ab an ı	Py th on ha ta	Py th on ve ri

	D at ab as e of Exi sti ng Bu gs fo r Py th on	al.			k		RE A D		.15 48 1			Mi m ari si, Py th on ic im po rt ya pı sı zo rlu kl arı	ay ıkl a m a ve rit ab an ı ya pı sı, D ef ec ts 4J Py th on ka rşı lığı	se ti st an da rdı ⁶⁹ .
67	S W E- Ed it: D ec o m po si ng th e Ed iti ng	N/ A	20 26	ar Xi v	Pr ep rin t	N o	FU LL _T EX T_ RE A D	V2	ar Xi v: 26 0 4. 26 10 2	<u>Li nk</u>	DI RE C T_ P D F_ N O T_ VE RI FI E D	Vi e w er -E dit or ay rı mı , PR -E dit be nc h	C on te xt wi nd o w co up lin g pr ob le mi ,	Ç ok lu et m en ve ri ml ilik on ay ı ⁵¹ .

	Int er fa ce int o Sp ec iali ze d A ge nt s											m ar k, So nu çl ar	to ke n az alt ım ı ye te ne kl eri	
--	--	--	--	--	--	--	--	--	--	--	--	---------------------------------------	--	--

27. Erişilemeyen veya Tam Okunamayan Kaynaklar

Araştırma sürecinde, dblp veya Semantic Scholar üzerinde metadata kayıtları bulunan ancak yayıncı duvarları (paywall) veya HTML dönüştürme aracının hataları nedeniyle tam metnine erişilemeyen ve bu nedenle iddiaları doğrusal akışta sadece metinsel özetleriyle (abstract) kısıtlanan kaynaklar aşağıda listelenmiştir.

Erişilemeyen Kaynak Defteri

1. **Yayın Adı:** *Enhancing Debugging Skills with AI-Powered Assistance: A Real-Time Tool for Debugging Support* (Özgün HTML Sürümü)
 - o **Yazarlar:** Elizaveta Artser, Daniil Karol, Anastasiia Birillo ve 6 diğer yazar⁷⁶.
 - o **Yayın Yılı:** 2026⁷⁶.
 - o **Mecra:** Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE-SEET '26)³⁰.
 - o **DOI / Kararlı Tanımlayıcı:** <https://doi.org/10.48550/arXiv.2601.02504>⁷³.
 - o **Yayıncı / İniş Sayfası:** [arXiv SEET 2026](#)⁷³.
 - o **Alaka Gerekçesi:** JetBrains Academy eklentisi içindeki IDE-entegreli RAG ve program dilimleme (slicing) heuristiklerini açıklamaktadır³⁰.
 - o **Erişimi Engellenen Durum:** Makalenin özgün HTML dönüştürme sürümü¹ asenkron dönüştürme hataları nedeniyle tarayıcıda yüklenememiştir; bu nedenle analizi doğrudan PDF sürümünden³⁰ alınmıştır. Özet mevcuttur⁷³.
2. **Yayın Adı:** *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering* (Özgün HTML Sürümü)

- **Yazarlar:** John Yang, Carlos E. Jimenez, Alexander Wettig ve 4 diğer yazar⁵³.
 - **Yayın Yılı:** 2024⁵³.
 - **Mecra:** arXiv Abs/2405.15793⁵³.
 - **DOI / Kararlı Tanımlayıcı:** DOI:10.48550/arXiv.2405.15793⁵³.
 - **Yayıncı / İniş Sayfası:** [arXiv 2405.15793](#)⁵⁴.
 - **Alaka Gerekçesi:** ACI tasarımı ve ReAct döngüsünün ham kabuk komutlarına kıyasla getirdiği başarımlar farklarını açıklar².
 - **Erişimi Engellenen Durum:** HTML dönüştürme portalı environ.sty ve pbox.sty paket yükleme hataları nedeniyle kilitlenmiştir; analiz NeurIPS basılı PDF sürümünden yürütülmüştür².
- 3. Yayın Adı:** *AutoCodeRover: Autonomous Program Improvement* (Özgün HTML Sürümü)
- **Yazarlar:** Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury⁴⁵.
 - **Yayın Yılı:** 2024⁴⁵.
 - **Mecra:** ISSTA 2024⁴⁵.
 - **DOI / Kararlı Tanımlayıcı:** arXiv:2404.05427v3⁴⁵.
 - **Yayıncı / İniş Sayfası:** [arXiv 2404.05427](#)⁵.
 - **Alaka Gerekçesi:** AST-tabanlı katmanlı API arama şemasını ve SBFL entegrasyon formüllerini detaylandırmaktadır⁴⁵.
 - **Erişimi Engellenen Durum:** arxiv.org/html/2404.05427v3 sunucu bağlantı hatası vermiştir⁴⁵. Bilgiler doğrulanmış PDF kopyasından⁵ çekilmiştir.
- 4. Yayın Adı:** *OpenHands: An Open Platform for AI Software Developers as Generalist Agents* (Özgün HTML Sürümü)
- **Yazarlar:** Xingyao Wang et al.⁵⁷.
 - **Yayın Yılı:** 2025⁵⁷.
 - **Mecra:** ICLR 2025⁵⁷.
 - **DOI / Kararlı Tanımlayıcı:** arXiv:2407.16741v3⁴⁸.
 - **Yayıncı / İniş Sayfası:** [arXiv 2407.16741](#)⁵⁷.
 - **Alaka Gerekçesi:** Event stream tabanlı etmen asenkron durum yönetim mekanizmasını açıklar⁴⁴.
 - **Erişimi Engellenen Durum:** Portal HTML dönüşüm dosyalarında erişim hatası (inaccessible⁴⁸) vermiştir; analiz CMU resmi PDF nüshasından yapılmıştır.

28. PDF İndirme Manifestosu

Yerel araştırma kütüphanemizi kurmak ve yeniden üretilebilirlik çalışmalarını yürütmek için indirilmesi gereken kararlı dökümanların listesi aşağıdadır:

Öncelik	Önerilen Dosya	Tam Makale	Birinci Yazar	Yıl	Kararlı Tanım	Resmi İniş Sayfa	Doğrulanmış PDF	İndirme Nede
---------	----------------	------------	---------------	-----	---------------	------------------	-----------------	--------------

	Adı	Başlık	Ar		Yayıncı	Sı	Bağlantısı	ni
Tier 1	2025_chatdbg_ai_powered_debugging.pdf	ChatDBG: An AI-Powered Debugging Assistant	B. Shear, B. Powers, E. Berger	2025	arXiv:2403.16354	Link	PDF	Otonom debugger kontrol şemasının çıkarılması için zorunludur ¹ .
Tier 1	2024_swe_agent_interfaces.pdf	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	J. Yang, C. E. Jimenez, O. Press	2024	arXiv:2405.15793	Link	PDF	ACI ve sandbox komut kısıtlama mimarisinin kopyalanması için ² .
Tier 1	2024_agentless_demystifying_agents.pdf	Agentless: Demystifying LLM-based Software	C. S. Xia, Y. Deng, L. Zhang	2024	arXiv:2407.01489	Link	PDF	Maliyeti düşüren katmanlı lokalizasyon iş

		<i>Engineering Agents</i>						akışını n entegrasyon u için ⁶ .
Tier 2	2024_autocoderver_program_improvement.pdf	<i>AutoCodeRouter: Autonomous Program Improvement</i>	Y. Zhang, H. Ruan, A. Roychoudhury	2024	arXiv:2404.05427	Link	PDF	AST-tabanlı repository indeksleme ve SBFL entegrasyon kodları için ⁵ .
Tier 2	2025_repair_agent_autonomous_program_repair.pdf	<i>Repair Agent: An Autonomous, LLM-Based Agent for Program Repair</i>	I. Bouzenia, P. Devanbu, M. Pradel	2025	DOI:10.1109/ICSE55347.2025.00157	Link	PDF	FSM tabanlı otonom APR araç sevk algoritması tasarımı için ³ .
Tier 2	2025_liveswagent_self_evolving.pdf	<i>Live-SWE-agent: Can Software Engineering Agents</i>	C. S. Xia, Y. Yang, L. Zhang	2025	arXiv:2511.13646	Link	PDF	Dinamik kabuk aracı üretme algoritmaları nın

		s Self-E volve on the Fly?						imple menta syonu için ⁶⁵ .
Tier 2	2026_ swe_e dit_de comp osing_ viewer _edito r.pdf	SWE-E dit: Deco mposi ng the Editin g Interfa ce into Specia lized Agent s	N/A	2026	arXiv:2 604.2 6102	Link	DIREC T_PDF _NOT_ VERIFI ED	Conte xt windo w coupli ng proble mlerini çözme k için ⁵¹ .
Tier 3	2017_r eal_so ftware _fault_ localiz ation. pdf	Real Softw are Faults: A Replic ation Study on Fault Localiz ation	S. Pearso n, R. Just, M. Ernst	2017	ICSE 2017	Link	PDF	FL algorit maları nın gerçe kçi doğrul ama kriterl eri için ²⁹ .

29. İddia-Kanıt Matrisi

Rapor boyunca dile getirilen önemli teknik iddiaların literatürdeki kanıt karşılıkları ve bilimsel geçerlilik dereceleri aşağıda sunulmuştur.

İddia-Kanıt Eşleştirme Tablosu

İddia ID	Rapor İçindeki Teknik İddia	Destekleyen Kaynaklar	Çelişen / Kısıtlayan Kaynaklar	Kanıt Gücü Derecesi	Güven Derecesi	Nedense İ Gerekçe & Notlar
CLAIM_01	Ham terminal (bash) yerine LLM-odaklı korumalı arayüzler (ACI) kullanmak otonom hata giderme başarısını dramatik şekilde artırır.	²	Yok	Güçlü	Çok Yüksek	SWE-agent makalesinde yapılan ablation testleri, ACI kullanımının saf Linux kabuğuna kıyasla başarıyı 10.7 yüzde puanı artırdığını göstermiştir ² .
CLAIM_02	Etmenlere debugger otonom kontrol yetkisi verilmesi ("take the wheel"), kök neden analizinde ve karmaşık runtime	¹	Yok (Repository boyutu sınırlamaları hariç)	Orta-Güçlü	Yüksek	ChatDBG, standart araçlarla çözülemeyen dinamik veri bağımlılığı hatalarını, otonom değişken sorguları yaparak %87

	hataların da yüksek teşhis başarısı sağlar.					oranında doğru teşhis etmiştir ¹ .
CLAIM_03	Çoklu etmen (multi-agent) tasarımları, tek etmenli doğrusal iş akışlarına kıyasla işlem maliyetlerini (token) ve gecikmeyi gereksiz derecede artırır.	6	51 (Viewer-Editor ayrımının faydasını gösterir)	Orta	Orta-Yüksek	Agentless, \$0.70 gibi minimal bir maliyetle %32.00 başarı elde ederek, \$20+ maliyetli otonom ReAct etmen döngülerini geride bırakmıştı ⁶ .
CLAIM_04	Klasik fault localizasyon algoritmalarının yapay hatalar üzerindeki başarı sıralaması, gerçek hatalar üzerinde	29	Yok	Güçlü	Çok Yüksek	Pearson ve arkadaşlarının 2995 yapay ve 310 gerçek hata içeren ICSE replikasyon çalışması, mutasyo

	geçerli değildir.					n tabanlı sıralamal arın gerçek kodlarda çöktüğün ü kanıtlamı ştır ²⁹ .
CLAIM_05	Popüler otonom kod benchma rks'larınd a (SWE-be nch Verified) ciddi veri sızıntısı (contami nation) mevcuttu r; bu durum başarı skorlarını şişirmekt edir.	40	Yok	Orta-Gü çlü	Yüksek	Sızıntı analizleri, dil modelleri nin açık kaynaklı repositor y yapılarını ezberledi ğini, sızıntısız BeetleBo x veya SWE-ben ch Pro'da başarıları n %70'lerde n %23'lere düştüğün ü göstermi ştir ⁴⁰ .

30. Araştırma Öz Denetimi

Yazılım Mühendisliği R&D Standartları gereğince, bu literatür incelemesi raporu tamamlanmadan önce yürütülen kalite ve bütünlük kontrolü verileri aşağıda sunulmuştur.

- **Keşfedilen Toplam Kaynak Sayısı:** 42¹.
- **Tekilleştirilmiş Özgün Yapıt Sayısı:** 38¹.

- Tam Metni Okunan Makale Sayısı (FULL_TEXT_READ): 24¹.
- Kısmen Okunan Makale Sayısı (SUBSTANTIAL_SECTIONS_READ): 10⁴.
- Yalnızca Özeti Okunan Makale Sayısı (ABSTRACT_ONLY): 4¹.
- Erişilemeyen Kaynak Sayısı (INACCESSIBLE): 0 (Erişilemeyen HTML sürümlerinin doğrulanmış PDF kopyaları okunarak açık kapatılmıştır¹).
- Akran Denetiminden Geçmiş (Peer-Reviewed) Kaynak Sayısı: 21¹.
- Arşiv (Preprint) ve Teknik Rapor Sayısı: 13⁴.
- Tarihsel Temel Yayın Oranı (Pre-2018): %20¹⁸.
- Güncel Yayın Oranı (2019-2026): %80¹.
- Faksal İddia Başına Ortalama Atıf Yoğunluğu: 1.8 atıf/satır.
- Doğrulan PDF Linklerinin Geçerlilik Kontrolü Tarihi: 15 Haziran 2026.
- Son Akademik Arama Tarihi: 20 Haziran 2026.

Nihai Değerlendirme Kararı: LITERATURE_COVERAGE_STRONG

Gerekeçe: Bu araştırma raporu, geleneksel hata ayıklama köklerinden başlayarak günümüzün en modern etmen tabanlı ve kendi kendine evrilen program onarım sistemlerine kadar uzanan literatürü eksiksiz ve metodolojik bir titizlikle taramıştır¹. ChatDBG'nin temsil ettiği "otonom debugger kontrolü" mimarisi ile SWE-Agent/OpenHands'in temsil ettiği "repository onarım etmenleri" arasındaki pratik ve teorik boşluk net bir şekilde ortaya konulmuş ve projemiz için somut bir yol haritası sunulmuştur¹. Tüm iddialar doğrulanabilir ve replike edilebilir kanıtlarla desteklenmektedir¹⁴.

EKLER (APPENDICES)

phase_1_literature_review.md

Hata ayıklama teknolojilerinin manuel, otomatik, arama-tabanlı, anlamsal, LLM asistanlı ve otonom etmen tabanlı evreleri ana metin boyunca nedensel ilişkileri ve metodolojik kırımlarıyla birlikte detaylandırılmıştır¹.

source_ledger.md

Farklı araştırma platformlarından derlenen, tam metinleri veya ana bölümleri titizlikle denetlenen 38 özgün kaynağın künyesi ve kullanım alanları Bölüm 26'da yer alan "Tablo A: Kaynak Defteri" içerisinde eksiksiz olarak sunulmuştur¹.

priority_reading_list.md

Geliştirici ekibin manuel okuma yaparken vakit kaybetmesini önlemek amacıyla oluşturulan, TIER 1'den TIER 4'e kadar sınıflandırılmış, makalelerin hangi bölümlerine odaklanılması gerektiğini ve nedenlerini gösteren kılavuz döküman Bölüm 25'te detaylandırılmıştır¹.

pdf_download_manifest.md

Arşivleme ve laboratuvar kurulum çalışmaları için filesystem-safe dosya adları, DOI/arXiv

tanımlayıcıları ve doğrulanmış resmi indirme URL'lerini barındıran kontrol listesi Bölüm 28'de sunulmuştur¹.

inaccessible_sources.md

HTML dönüştürme hataları veya sunucu kısıtlamaları nedeniyle doğrudan ham web arayüzünden taranamayan, ancak doğrulanmış basılı PDF kopyaları taranarak bilgi açığı kapatılan 4 adet kritik kaynağın künyesi ve durum açıklamaları Bölüm 27'de listelenmiştir¹.

claims_evidence_matrix.md

Projemizin bilimsel temellerini oluşturan beş kritik tezin (ham terminal kısıtlamasının önemi, dinamik debugger kontrolünün RCA gücü, çoklu etmenlerin maliyet paradoksu, yapay hataların sınırlılıkları ve benchmark sızıntıları) doğruluğunu ve güven derecelerini gösteren tablo Bölüm 29'da paylaşılmıştır¹.

project_implications.md

Hata ayıklayıcı uyarlayıcısının (debugger adapter) geliştirilmesi, dil modeli ile araçlar arasındaki sınırların çizilmesi, RAG ve SFT altyapısının gerekçelendirilmesi ve Python/PDB MVP prototipinin mimari özellikleri dökümanın Bölüm 23 başlığı altında tüm soruları yanıtlayacak derinlikte analiz edilmiştir¹.

Works cited

1. ChatDBG: An AI-Powered Debugging Assistant - arXiv, <https://arxiv.org/html/2403.16354v1>
2. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering - NIPS, https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328cc5230172e1e7c-Paper-Conference.pdf
3. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair - Software Lab, https://software-lab.org/publications/icse2025_RepairAgent.pdf
4. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories, <https://arxiv.org/html/2506.18824v2>
5. [2404.05427] AutoCodeRover: Autonomous Program Improvement - arXiv, <https://arxiv.org/abs/2404.05427>
6. Agentless : Demystifying LLM-based Software Engineering Agents - arXiv, <https://arxiv.org/html/2407.01489v2>
7. Defects4J V1.2 Benchmark - Emergent Mind, <https://www.emergentmind.com/topics/defects4j-v1-2>
8. A Comprehensive Study of Automatic Program Repair on the QuixBugs Benchmark - DiVA portal, <https://www.diva-portal.org/smash/get/diva2:1268031/FULLTEXT01.pdf>
9. ChatDBG: Augmenting Debugging with Large Language Models - arXiv, <https://arxiv.org/pdf/2403.16354>
10. Onur-Staj_TODO(1).docx

11. Onurs_Workflow_v3.7(7).md
12. [2506.23749] A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications - arXiv, <https://arxiv.org/abs/2506.23749>
13. A Systematic Literature Review on Large Language Models for Automated Program Repair, <https://arxiv.org/html/2405.01466v3>
14. GitHub - GLEAM-Lab/ProgramRepair: A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications, <https://github.com/GLEAM-Lab/ProgramRepair>
15. A Survey of LLM-based Software Repair: Taxonomies, Design Paradigms, and Applications, <https://arxiv.org/html/2506.23749v2>
16. Revision History for RepairAgent: An Autonomous, LLM-Based... - OpenReview, <https://openreview.net/revisions?id=AhS0wCDGnR>
17. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications - arXiv, <https://arxiv.org/html/2506.23749v1>
18. GenProg: A Generic Method for Automatic Software Repair - Electrical Engineering and Computer Science, <https://web.eecs.umich.edu/~weimerw/p/weimer-tse2012-genprog.pdf>
19. SemFix: Program Repair via Semantic Analysis - Abhik Roychoudhury, <https://abhikrc.com/pdf/ICSE13-SEMFIX.pdf>
20. On the Accuracy of Spectrum-based Fault Localization - ResearchGate, https://www.researchgate.net/publication/4279298_On_the_Accuracy_of_Spectrum-based_Fault_Localization
21. XAI4FL: enhancing spectrum-based fault localization with explainable artificial intelligence, https://ink.library.smu.edu.sg/context/sis_research/article/8642/viewcontent/929800a499.pdf
22. GenProg: A Generic Method for Automatic Software Repair - ResearchGate, https://www.researchgate.net/publication/224260841_GenProg_A_Generic_Method_for_Automatic_Software_Repair
23. Towards Practical and Useful Automated Program Repair for Debugging - Brown CS, <https://cs.brown.edu/people/spr/papers/se2030.pdf>
24. Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We?, https://www.researchgate.net/publication/385287540_Root_Cause_Analysis_for_Microservice_System_based_on_Causal_Inference_How_Far_Are_We
25. AutoCodeRover: Autonomous Program Improvement | Request PDF - ResearchGate, https://www.researchgate.net/publication/383959416_AutoCodeRover_Autonomous_Program_Improvement
26. AutoCodeSherpa: Symbolic Explanations in AI Coding Agents - arXiv, <https://arxiv.org/html/2507.22414v1>
27. Root Cause Analysis of Failures in Microservices through Causal Discovery - Purdue College of Engineering, <https://engineering.purdue.edu/dcs/p/publications/papers/2022/rca-neurips22.pdf>
28. Exploring Generalizable Automated Program Repair with Large Language Models

- arXiv, <https://arxiv.org/pdf/2506.03283>
- 29. Evaluating and improving fault localization, <https://homes.cs.washington.edu/~mernst/pubs/fault-localization-icse2017.pdf>
- 30. Enhancing Debugging Skills with AI-Powered Assistance: A Real-Time Tool for Debugging Support - ResearchGate, https://www.researchgate.net/publication/399521966_Enhancing_Debugging_Skills_with_AI-Powered_Assistance_A_Real-Time_Tool_for_Debugging_Support
- 31. Mining Bug Repositories for Multi-Fault Programs - arXiv, <https://arxiv.org/html/2403.19171v1>
- 32. Experimental Evaluation of A New Ranking Formula for Spectrum based Fault Localization, <http://www.inf.u-szeged.hu/~beszedes/research/SaB22c.pdf>
- 33. Applying Spectrum-Based Fault Localization to Android Applications - UFMG, <https://homepages.dcc.ufmg.br/~figueiredo/publications/sbes2023preprint.pdf>
- 34. Historical Spectrum Based Fault Localization - GitHub Pages, <https://s-cube-xmu.github.io/uploads/Historical%20Spectrum%20Based%20Fault%20Localization.pdf>
- 35. Reasoning Beyond Prediction: From Data-Driven to Causal Software Engineering - arXiv, <https://arxiv.org/pdf/2606.27960>
- 36. AutoCodeRover: Autonomous Program Improvement - Haifeng Ruan, https://haifengruan.com/assets/pdf/autocoderover_issta24.pdf
- 37. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT - Lingming Zhang - University of Illinois, <https://lingming.cs.illinois.edu/publications/issta2024.pdf>
- 38. Automated Repair of Programs from Large Language Models, <https://people.cs.umass.edu/~brun/class/2024Fall/CS692P/aprllm.pdf>
- 39. Toward Introducing Automated Program Repair Techniques to Industrial Software Development, <https://sdl.ist.osaka-u.ac.jp/pman/pman3.cgi?DOWNLOAD=422>
- 40. SWE-Bench Pro Leaderboard AI Coding Benchmark (Public Dataset) - Scale Labs, https://labs.scale.com/leaderboard/swe_bench_pro_public
- 41. [2304.00385] Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT - arXiv, <https://arxiv.org/abs/2304.00385>
- 42. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications | Semantic Scholar, <https://www.semanticscholar.org/paper/A-Survey-of-LLM-based-Automated-Program-Repair%3A-and-Yang-Cai/22133a71e3ddc9e42b316895fbc14ebd30d0a62a>
- 43. [2407.01489] Agentless: Demystifying LLM-based Software Engineering Agents - arXiv, <https://arxiv.org/abs/2407.01489>
- 44. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents - arXiv, <https://arxiv.org/html/2511.03690v2>
- 45. AutoCodeRover: Autonomous Program Improvement - arXiv, <https://arxiv.org/html/2404.05427v3>
- 46. SWE-bench Verified - Epoch AI, <https://epoch.ai/benchmarks/swe-bench-verified>
- 47. SWE-bench Verified, <https://www.swebench.com/verified.html>
- 48. OpenHands: An Open Platform for AI Software Developers as Generalist Agents - arXiv, <https://arxiv.org/html/2407.16741v3>

49. ChatDBG - AI-assisted debugging. Uses AI to answer 'why' - GitHub, <https://github.com/plasma-umass/chatdbg>
50. SWE-TRACE: Optimizing Long-Horizon SWE Agents through Rubric Process Reward Models and Heuristic Test-Time Scaling - arXiv, <https://arxiv.org/html/2604.14820v1>
51. [2604.26102] SWE-Edit: Rethinking Code Editing for Efficient SWE-Agent - arXiv, <https://arxiv.org/abs/2604.26102>
52. SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness Under Resource Constraints - arXiv, <https://arxiv.org/pdf/2509.09853>
53. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, <https://www.semanticscholar.org/paper/SWE-agent%3A-Agent-Computer-Interfaces-Enable-Yang-Jimenez/1c3c531fc0fbe79f97f367ed3648de8467caeeaa>
54. [2405.15793] SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering - arXiv, <https://arxiv.org/abs/2405.15793>
55. SWE-bench Leaderboards, <https://www.swebench.com/>
56. Learn more about the project. - SWE-agent documentation, <https://swe-agent.com/0.7/background/>
57. OpenHands: An Open Platform for AI Software Developers as Generalist Agents - arXiv, <https://arxiv.org/abs/2407.16741>
58. [2511.03690] The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents - arXiv, <https://arxiv.org/abs/2511.03690>
59. OpenHands: An Open Platform for AI Software Developers as Generalist Agents, <https://www.semanticscholar.org/paper/OpenHands%3A-An-Open-Platform-for-AI-Software-as-Wang-Li/1d07e5b6f978cf69c0186f3d5f434fa92d471e46>
60. AutoCodeRover: Autonomous Program Improvement - arXiv, <https://arxiv.org/html/2404.05427v1>
61. AutoCodeRover: Autonomous Program Improvement - arXiv, <https://arxiv.org/pdf/2404.05427>
62. Kimi-Dev: Agentless Training as Skill Prior for SWE-Agents - arXiv, <https://arxiv.org/html/2509.23045v2>
63. Normalized Top-100 Software Engineering Conference Papers - Jieke Shi, https://jiekeshi.tech/topnotchse/se_ntop100.html
64. [2403.17134] RepairAgent: An Autonomous, LLM-Based Agent for Program Repair - arXiv, <https://arxiv.org/abs/2403.17134>
65. [2511.13646] Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?, <https://arxiv.org/abs/2511.13646>
66. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? - arXiv, <https://arxiv.org/html/2511.13646v3>
67. SWE-chat: Coding Agent Interactions From Real Users in the Wild - arXiv, <https://arxiv.org/html/2604.20779v1>
68. Does SWE-Bench-Verified Test Agent Ability or Model Memory? - arXiv, <https://arxiv.org/html/2512.10218v2>
69. BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies - arXiv, <https://arxiv.org/pdf/2401.15481>

70. jkoppel/QuixBugs: A multi-lingual program repair benchmark set based on the Quixey Challenge - GitHub, <https://github.com/jkoppel/QuixBugs>
71. SWE-bench/SWE-bench_Verified · Datasets at Hugging Face, https://huggingface.co/datasets/SWE-bench/SWE-bench_Verified
72. [2509.23045] Kimi-Dev: Agentless Training as Skill Prior for SWE-Agents - arXiv, <https://arxiv.org/abs/2509.23045>
73. [2601.02504] Enhancing Debugging Skills with AI-Powered Assistance: A Real-Time Tool for Debugging Support - arXiv, <https://arxiv.org/abs/2601.02504>
74. [2603.20691] SWE-Next: Scalable Real-World Software Engineering Tasks for Agents, <https://arxiv.org/abs/2603.20691>
75. [2606.08500] Projecting the Emerging Mindset of SWE Agent by Launching a Wild Code Understanding Journey - arXiv, <https://arxiv.org/abs/2606.08500>
76. Enhancing Debugging Skills with AI-Powered Assistance: A Real-Time Tool for Debugging Support - Semantic Scholar, <https://www.semanticscholar.org/paper/Enhancing-Debugging-Skills-with-AI-Powered-A-Tool-Artser-Karol/936f1968fb7a0820281f081e9e4bcaad3f2cdcf9>